第四部分

算法设计策略——动态规划

主讲人：孙歧峰

联系电话：13864717907
Q Q号码：274342509
邮　　箱：sunqf@upc.edu.cn

算法设计策略——动态规划（Dynamic Programming）

上一节，我们学习了最优化问题的一种算法设计策略，它的基本思想是设计出具有局部特征的“贪心准则”，然后通过多次贪心选择，使问题得以解决，得到问题的全局最优解。

这是一种思想简单、也很自然的设计策略。关键是要设计出“贪心准则”，而且要证明这种“贪心”是一定可以导致全局最优的。这也是难点！

贪心策略要保证有效：

（1）问题具有最优子结构性质；

（2）满足贪心选择性质；

那么，当贪心遇到麻烦时，怎么办？问题用什么策略求解？



例如，看下面带权活动安排问题：

假设有某种资源，一个活动在某一时间段可以占用（使用）该资源，每个活动占用资源的时间为 $[s_i,f_i)$，另外每个活动有一个权值w_i（可以定义为活动收益）。现要求总收益最大的相容活动子集。

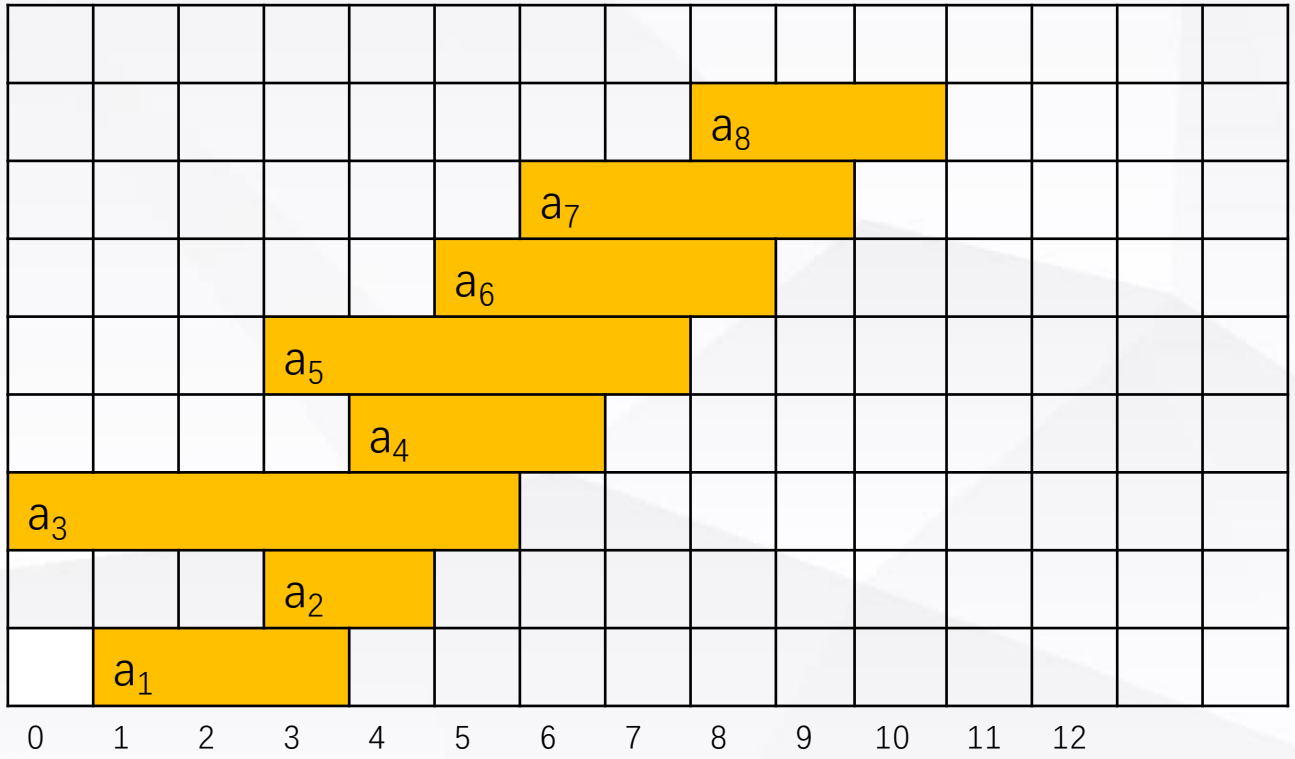
如果各个活动权值都相同，就是最普通的最大相容子集问题。该问题有贪心算法，贪心准则：在相容的前提下，最早结束的活动先安排！

但是，对于带权的活动安排问题，显然该贪心准则是不行的。例如：

算法设计策略——动态规划（Dynamic Programming）

a_i	1	2	3	4	5	6	7	8
s_i	1	3	0	4	3	5	6	8
f_i	4	5	6	7	8	9	10	11
w_i	12	20	23	13	26	20	11	16

https://blog.csdn.net/qq_3765718



不带权值：
安排3个活动：a1,a4,a8
权值和=41

带权值：
安排2个活动：a5,a8
权值和=42

算法设计策略——动态规划（Dynamic Programming）

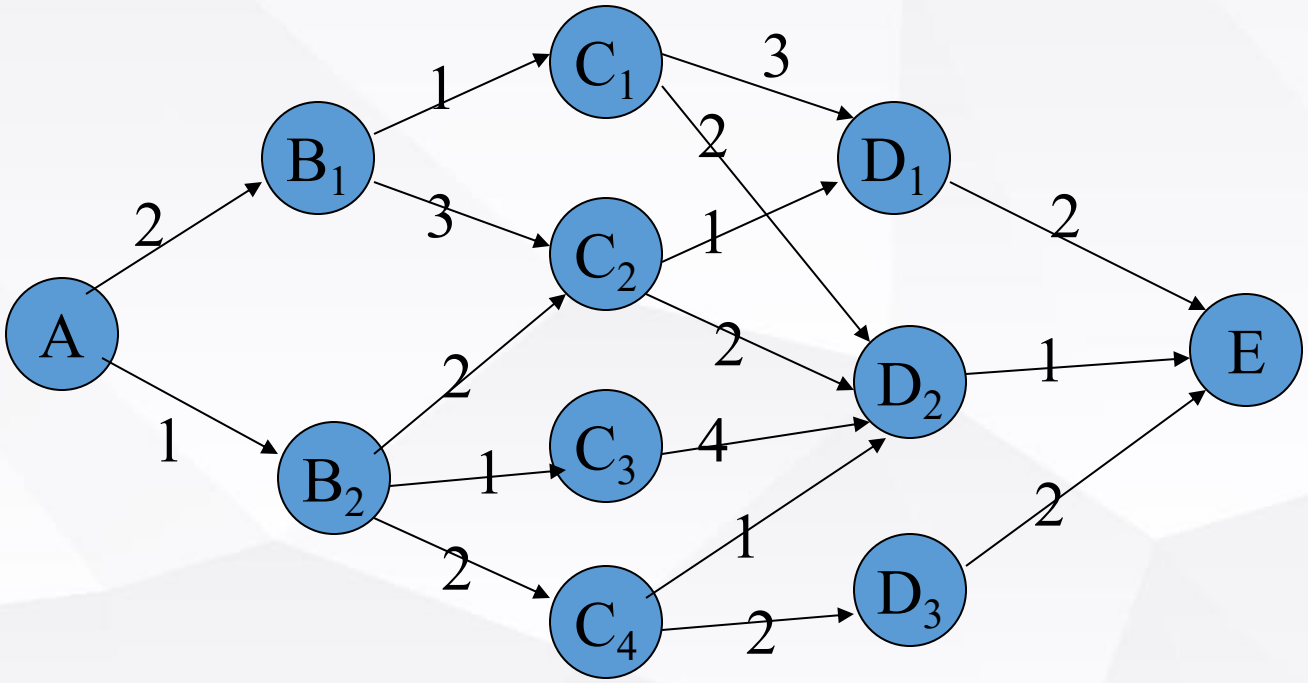
动态规划方法是处理分段过程最优化问题的一类极其有效的方法。20世纪50年代由美国数学家Richard Bellman发明的。“programing”：计划、规划。

在实际生活中，有一类问题的活动过程可以分成若干个阶段，而且在任一阶段后的行为依赖于该阶段的状态，而与该阶段之前的过程如何达到这种状态的方式无关。这类问题的解决是多阶段的决策过程。

算法设计策略——动态规划（Dynamic Programming）

看一个例子：最短路径问题

现有一张地图，各结点代表城市，两结点间连线代表道路，线上数字表示城市间的距离。如下图所示，试求出从结点A到结点E的最短距离。



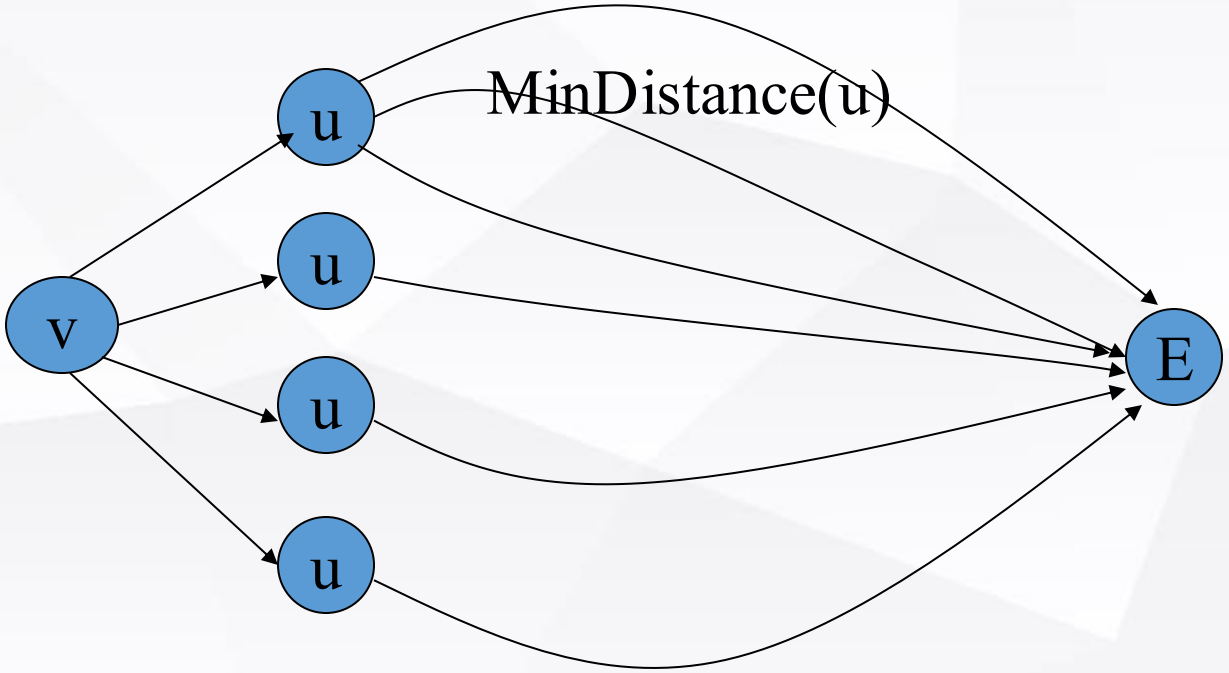
算法设计策略——动态规划（Dynamic Programming）

我们可以用深度优先搜索法来解决此问题，该问题的递归式为

$$MinDistance(v) = \min_{u \in \delta(v)} \{w(v,u) + MinDistance(u)\}$$

其中 $\delta(v)$是与v相邻的结点的集合，w(v,u)表示从v到u的边的长度。即：

v到E的最短路径 = min{ 边<v,u>长度 + 其各个后继u到E的最短路径}



要注意：上面的公式刻画了子问题最优解和全局最优解的关系！

算法设计策略——动态规划（Dynamic Programming）

很容易写出一个递归算法，如下：

```
int  MinDistance(v)
  {
    if(v==E) return 0
    else { min=maxint;
           for(所有没有访问过的顶点vi)
           if(v和vi相邻接)
            {   标记vi访问过了;
                t=v到vi的距离w(v,vi)+MinDistance(vi);
                标记vi未访问过;
                if(t<min) min=t; }
          }
   }
```

开始时标记所有的顶点未访问过，MinDistance(A)就是从A到E的最短距离。

算法设计策略——动态规划（Dynamic Programming）

对于上例，按照算法的计算过程为：

$D(A)=\min\{\ 2+D(B_1)\ ,\ 1+D(B_2)\ \}=\min\{\ 2+4\ ,\ 1+4\ \}=5$

$D(B_1)=\min\{\ 1+D(C_1)\ ,\ 3+D(C_2)\ \}=\min\{\ 1+3\ ,\ 3+3\ \}=4$

$D(B_2)=\min\{\ 2+D(C_2)\ ,\ 1+D(C_3)\ ,\ 2+D(C_4)\ \}=\min\{\ 2+3\ ,\ 1+5\ ,\ 2+2\ \}=4$

$D(C_1)=\min\{\ 3+D(D_1)\ ,\ 2+D(D_2)\ \}=\min\{\ 3+2\ ,\ 2+1\ \}=3$

$D(C_2)=\min\{\ 1+D(D_1)\ ,\ 2+D(D_2)\ \}=\min\{\ 1+2\ ,\ 2+1\ \}=3$

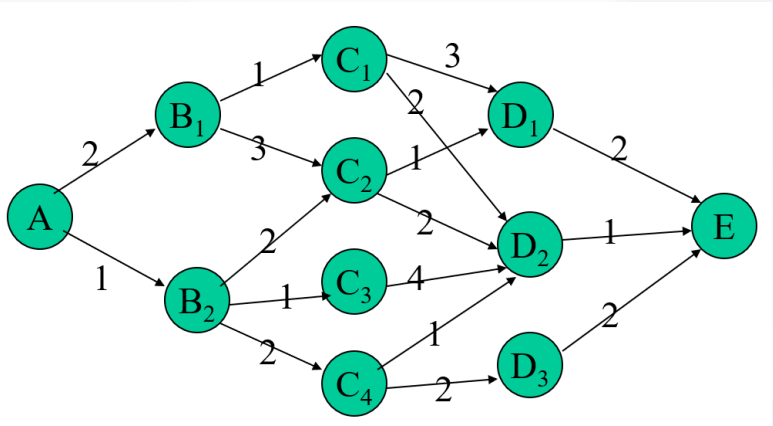
$D(C_3)=\min\{\ 4+D(D_2)\ \}=\min\{\ 4+1\ \}=5$

$D(C_4)=\min\{\ 1+D(D_2)\ ,\ 2+D(D_3)\ \}=\min\{\ 1+1\ ,\ 2+2\ \}=2$

$D(D_1)=\min\{\ 2+D(E)\ \}=2+0=2$

$D(D_2)=\min\{\ 1+D(E)\ \}=1+0=1$

$D(D_3)=\min\{\ 2+D(E)\ \}=2+0=2$



D(E)=0　　最短路径长度：5

路径轨迹：A→B2 → C4 → D2 → E

算法设计策略——动态规划（Dynamic Programming）

这个算法的效率如何呢？我们可以看到，每次除了已经访问过的城市外，其他城市都要访问，所以时间复杂度为O(n!)，这是一个“指数级”的算法.

问题出在那里?

首先，我们来观察一下这个算法。在求从B_1到E的最短距离的时候，先求出从C_2到E的最短距离；而在求从B_2到E的最短距离的时候，又求了一遍从C_2到E的最短距离。也就是说，从C_2到E的最短距离我们求了两遍。同样可以发现，在求从C_1、C_2到E的最短距离的过程中，从D_1到E的最短距离也被求了两遍。而从D_2到E的最短距离被求了四遍。

问题在于，划分出的很多的子问题是重叠的！那么如何解决这类问题呢?

一种解决办法是： 将每次求出的从某顶点v到E的最短距离记录下来，在算法中递归地求MinDistance(v)时先检查以前是否已经求过了MinDistance(v)，如果求过了，则不用重新求一遍，只要查找以前的记录就可以了。这样，由于所有的顶点有n个，因此不同的状态数目有n个，该算法的数量级为O(n)。 —— 备忘录

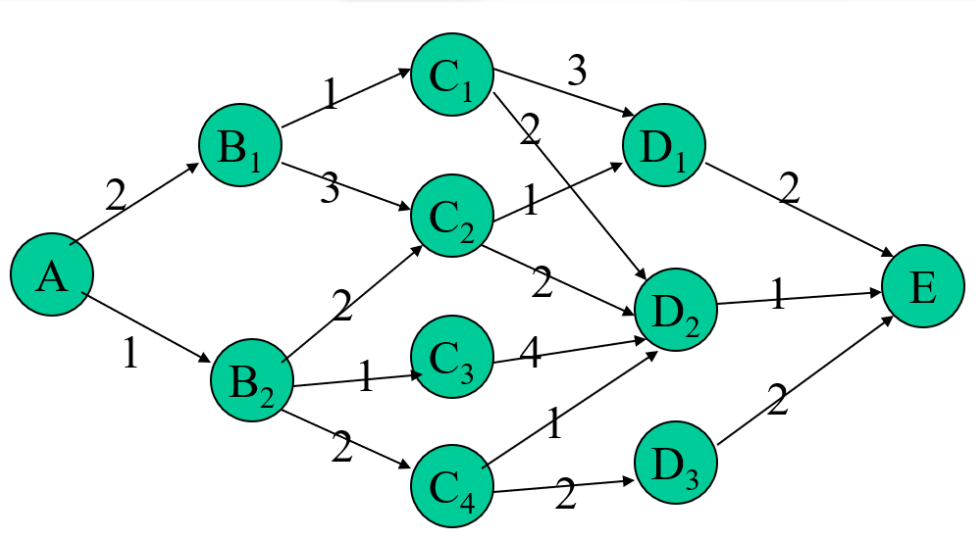
算法设计策略——动态规划（Dynamic Programming）

另一种解决办法是：

可以将这种递归优化为递推(先求解简单问题)，用已经计算出来的结果计算后面的，即按一定顺序求解子问题。

看图，可以发现，A只和B_i相邻，B_i只和C_i相邻,...，依此类推。这样，我们可以将原问题的解决过程划分为4个阶段，设$S_1=\{A\}$, $S_2=\{B_1, B_2\}$, $S_3=\{C_1, C_2, C_3, C_4\}$, $S_4=\{D_1, D_2, D_3\}$，

$F_k(u)$表示从S_k中的点u到E的最短距离，δ(u)表示与u邻接的顶点集合。则：

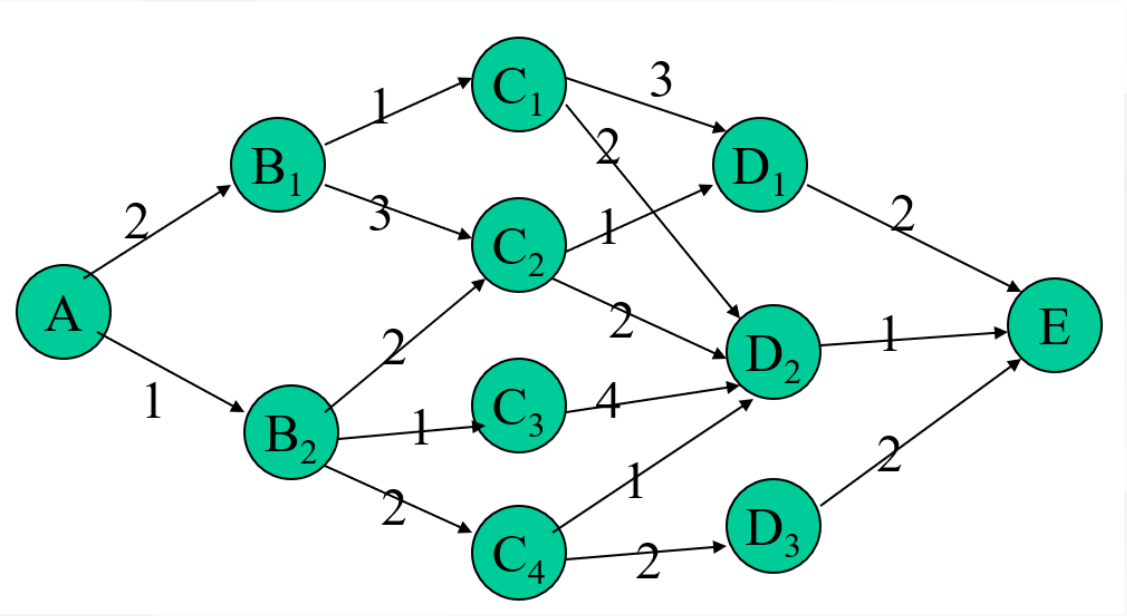


算法设计策略——动态规划（Dynamic Programming）

$$F_k(u) = \min_{u \in S_k, v \in S_{k+1} \cap \delta(u)} \{w(u,v) + F_{k+1}(v)\}$$

并且有边界条件：　　$F_5(E) = 0$

显然，可以递推地求出$F_1(A)$，即A到E的最短距离。



算法设计策略——动态规划（Dynamic Programming）

$F_1(u)=\min\{w(u,v)+F_2(v)\}$

$u=\{A\}, v=\{B_1, B_2\}$

$F_2(u)=\min\{w(u,v)+F_3(v)\}$

$u=\{B_1, B_2\}, v=\{C_1, C_2, C_3, C_4\}$

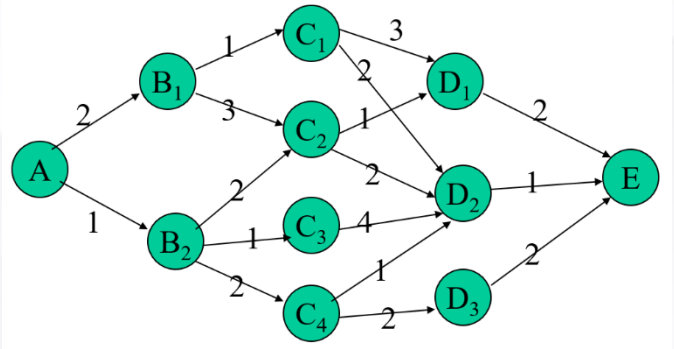
$F_3(u)=\min\{w(u,v)+F_4(v)\}$

$u=\{C_1, C_2, C_3, C_4\}, v=\{D_1, D_2, D_3\}$

$F_4(u)=\min\{w(u,v)+F_5(v)\}$

$u=\{D_1, D_2, D_3\}$ $v=\{E\}$

$F_5(E)=0$



得到递推过程如下：

$F_1(A)=\min\{2+4, 1+4\}=5$

$F_2(B_1)=\min\{1+3, 3+3\}=4$

$F_2(B_2)=\min\{2+3, 1+5, 2+2\}=4$

$F_3(C_1)=\min\{3+2, 2+1\}=3$

$F_3(C_2)=\min\{1+2, 2+1\}=3$

$F_3(C_3)=\min\{4+1\}=5$

$F_3(C_4)=\min\{1+1, 2+2\}=2$

$F_4(D_1)=\min\{2+0\}=2$

$F_4(D_2)=\min\{1+0\}=1$

$F_4(D_3)=\min\{2+0\}=2$

算法设计策略——动态规划（Dynamic Programming）

具体算法如下：

```
DynamicProgramming;
{ F5[E]=0;
   for(k=4;k>=1;k--}   //4个阶段，从后面开始前推
     for(each u ∈Sk)    //对该阶段的每个状态
      {    //Fk(u)表示从Sk中的点u到E的最短距离，
           //δ(u)表示与u邻接的顶点集合。
          Fk[u]=无穷大;
          for(each v∈Sk+1∩δ(u) )
          if(Fk[u]>w(u,v)+Fk+1[v])
            Fk[u]=w(u,v)+Fk+1[v];
      };
   输出F1[A]; };
```

时间复杂度为O(n)。
因为所有的状态数（顶点）为n，每个只访问1次。

这种高效算法的设计方法，就是*动态规划的策略。*

算法设计策略——动态规划（Dynamic Programming）

在这个例子中，有这样一个特点：

一个最短路径问题可变成多段判决问题，利用了最短路径的一个性质：从起点到终点的最短路径也是该路径上各点到终点的最短路径。 ——最优子结构性质

对于具有n个输入的最优解问题，它们的活动过程往往划分为若干个阶段，每一个阶段的决策依赖于前一阶段的状态，由决策所采取的动作使状态发生转移，成为下一阶段的决策依据。如下图所示。S_0 是初始状态，依据此状态做出决策P_1，按照P_1所采取的动作，使状态转换为S_1,…,经过一系列的决策和状态转移，到达最终状态S_n。于是一个决策序列就在不断变化的状态中产生。

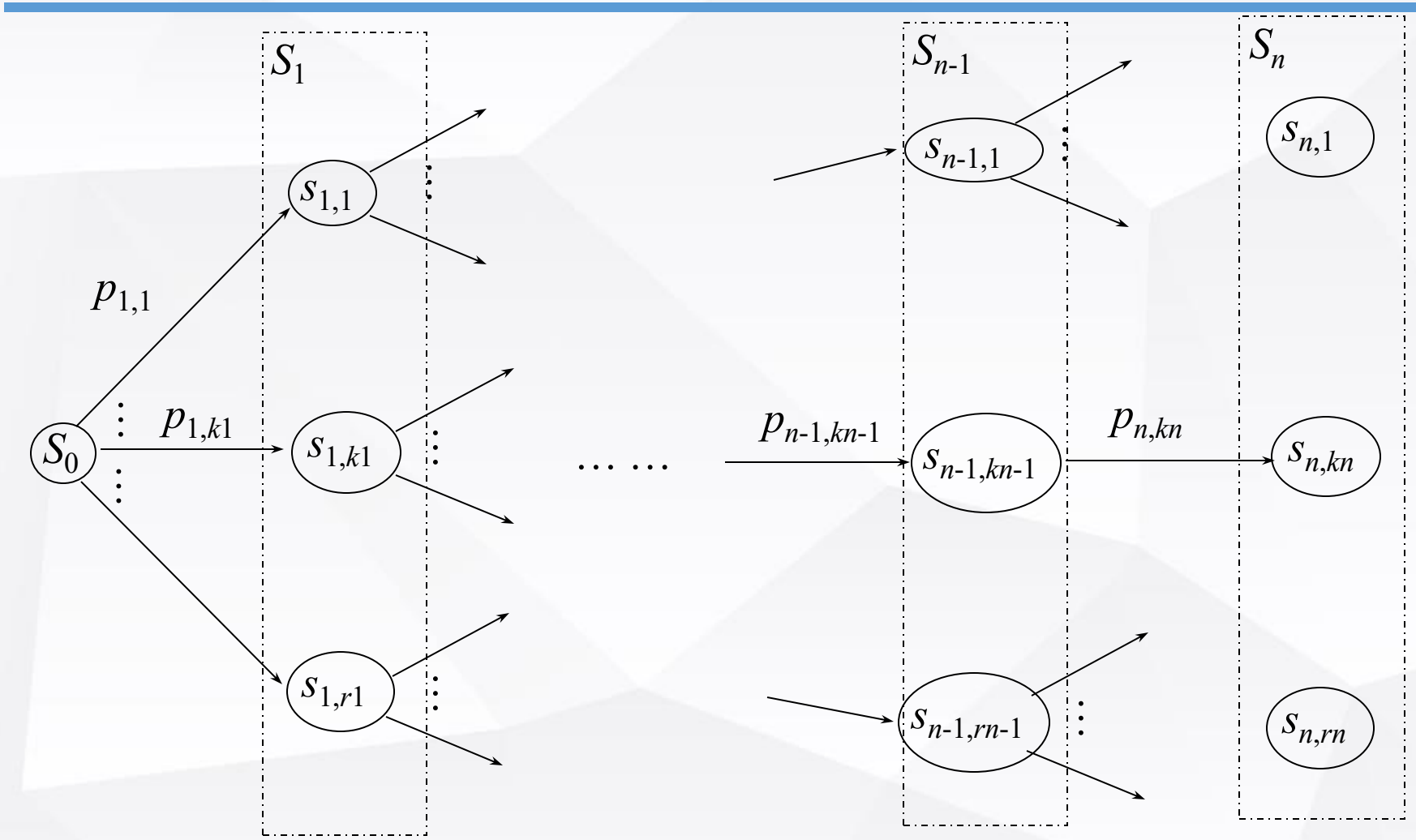
$$S_0 \xrightarrow{P_1} S_1 \xrightarrow{P_2} S_2 \quad \cdots\cdots\cdots \quad S_{n-1} \xrightarrow{P_n} S_n$$

由于每一阶段的决策仅与前一阶段所产生的状态有关，而与如何达到这种状态的方式无关。因此，可以把每一阶段作为一个子问题来处理。决策过程的每一个阶段，都可能有多种决策可以选择，其中只有一个决策对全局是最优的。

算法设计策略——动态规划（Dynamic Programming）

假设对一种状态，可以作出多种决策，而每一种决策可以产生一种新的状态。在这种假设下，根据最优化原理，对于初始状态S_0，$P_1=\{p_{1,1},p_{1,2},\cdots,p_{1,r1}\}$是可能的决策集合，由它们产生的状态$S_1=\{s_{1,1},s_{1,2},\cdots,s_{1,r1}\}$，其中$s_{1,k}$是对应于决策$p_{1,k}$所产生的状态。但是此时尚无法判定哪一个决策是最优的，于是，把这些决策值集合作为这一阶段的子问题的解保存起来，依次类推，在状态集合S_1上作出的决策集合P_2，产生了状态集合S_2。最后，对于状态$S_{n-1}=\{s_{n-1,1},s_{n-1,2},\cdots s_{n-1,rn-1}\}$，$P_n=\{p_{n,11},\cdots p_{n,1k1},p_{n,21},\cdots,p_{n,2k2},\cdots,p_{n,rn-11},\cdots,p_{n,n-1,kn-1}\}$是可能的决策集合，其中决策值集合$\{p_{n,t1},\cdots p_{n,ti}\}$是依据状态$s_{n-1,t}$所作出的可能的决策。

算法设计策略——动态规划（Dynamic Programming）

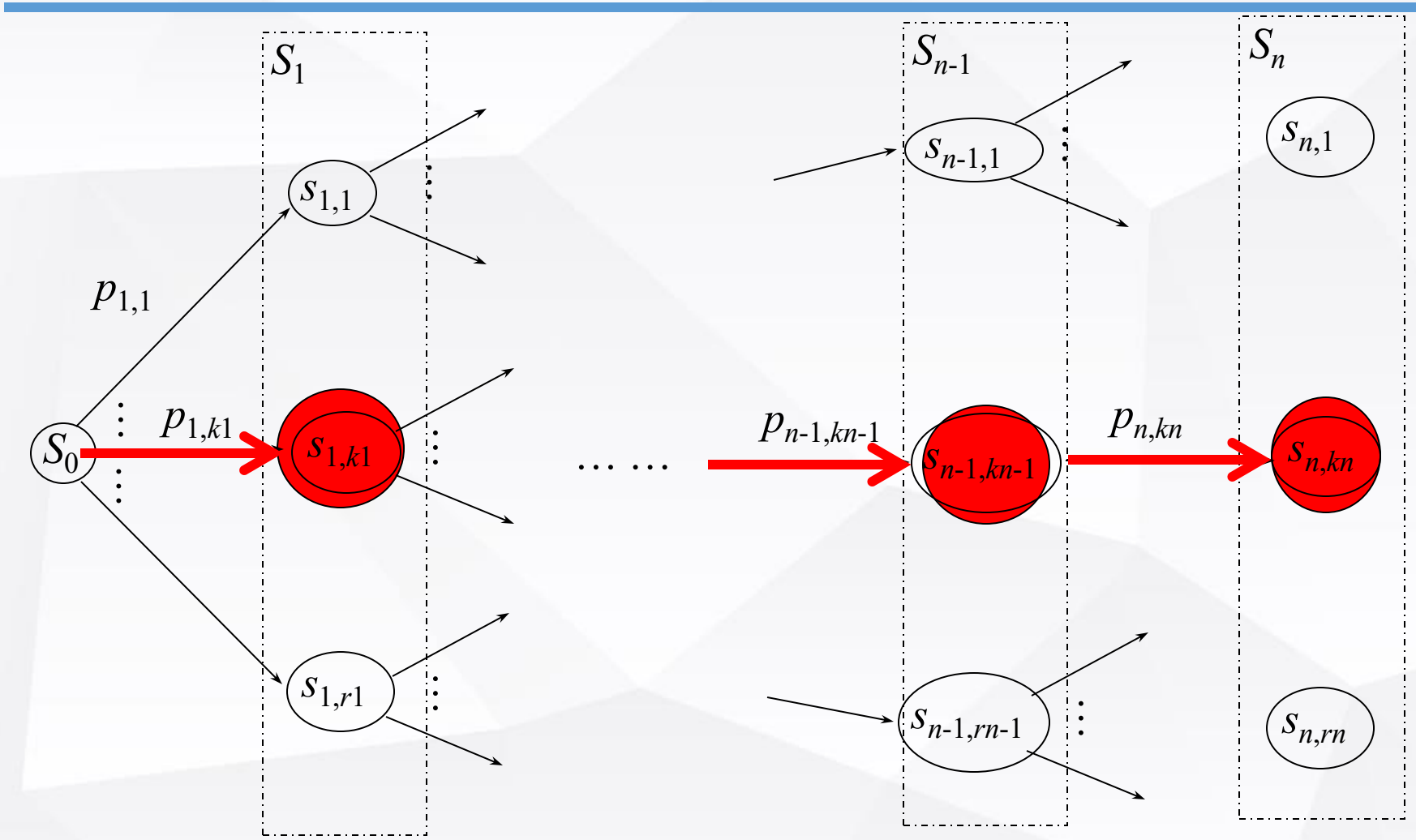


动态规划的决策过程

算法设计策略——动态规划（Dynamic Programming）

由P_n所产生的状态集合$S_n=\{s_{n,11},\cdots,s_{n,1k1},\cdots,s_{n,21},\cdots s_{n,2k2},\cdots,s_{n,rn-11},\cdots,s_{n,rn-1kn-1}\}$是最终的状态集合，其中*只有一个状态是最优的*。假设这个状态是$S_{n,kn}$，它由决策$P_{n,kn}$所产生，由此，可以确定$p_{n,kn}$是最优决策。假定，$P_{n,kn}$是依据状态$S_{n-1,kn-1}$作出的，由此回溯，使状态达到$S_{n-1,kn-1}$的决策是最优决策，这种回溯一直进行到$p_{1,k1}$，从而得到一个最优决策序列$\{p_{1,k1},p_{2,k2},\cdots,p_{n,kn}\}$，而这个决策序列导致了状态转移$\{s_0,s_{1,k1},s_{2,k2},\cdots,s_{n,kn}\}$。根据最优化原理，上述决策序列是依据初始状态$S_0$和初始决策$p_{1,k1}$所产生的状态而构成的一个最优决策序列。由这个决策序列，导致了由初始状态 S_0到最优状态$S_{n,kn}$的转移。

算法设计策略——动态规划（Dynamic Programming）



动态规划的决策过程

算法设计策略——动态规划（Dynamic Programming）

在上述的决策过程中，有一个赖以决策的策略或目标，把这种策略或目标称为动态规划函数。它由问题的性质和特点所确定，并应用于每一个阶段的决策。因此，整个决策过程，可以递归地进行，或以循环迭代的方法进行。这样，动态规划函数可以递归地定义，也可以用递推公式来表达。

上面非形式地叙述了动态规划的决策过程。可以看到，最优决策是在最后阶段形成的，然后向前倒推，直到初始阶段；而决策的具体结果及所产生的状态转移，却是由初始阶段开始进行计算的，然后向后递归或迭代，直到最终结果。

下面介绍一些基本的概念

算法设计策略——动态规划（Dynamic Programming）

一、动态规划的基本概念

1. 多阶段决策问题的定义

*多阶段决策过程，*是指这样的一类特殊的活动过程，问题可以按时间顺序分解成若干相互联系的阶段，在每一个阶段都要做出决策，全部过程的决策是一个决策序列。要使整个活动的总体效果达到最优的问题，称为*多阶段决策问题。*例如：上面我们提到的最短路径问题就是一个多阶段决策问题。

例1 ： 生产计划问题1

工厂生产某种产品，每单位(千件)的成本为1(千元)，每次开工的固定成本为3(千元)，工厂每季度的最大生产能力为6(千件)。经调查，市场对该产品的需求量第一、二、三、四季度分别为 2，3，2，4(千件)。如果工厂在第一、二季度将全年的需求都生产出来，自然可以降低成本(少付固定成本费)，但是对于第三、四季度才能上市的产品需付存储费，每季每千件的存储费为0.5(千元)。还规定年初和年末这种产品均无库存。试制订一个生产计划，即安排每个季度的产量，使一年的总费用(生产成本和存储费)最少。

算法设计策略——动态规划（Dynamic Programming）

	第1季度	第2季度	第3季度	第4季度
需求：	p_1：	p_2：	p_3：	p_4：
安排产量：	x_1：	x_2：	x_3：	x_4：
固定成本：	c_1：	c_2：	c_3：	c_4：
储存成本：	s_1：	s_2：	s_3：	s_4：

$c_i=3+x_i*1$ if $x_i>0$ $x_i<=6$

$s_i=(x_i-p_i)*0.5$ if$(x_i-p_i)>0$

Min $s=(c_1+c_2+c_3+c_4)+(s_1+s_2+s_3+s_4)$

例2： 生产计划问题2

某工厂购进1000台机器，准备生产P_1、P_2两种产品，若生产产品P_1，每台机器可年收入50千元，损坏率达65%；若生产产品P_2，每台机器年收入为40千元，但损坏率仅有40%；估计三年后将有新机器出现，旧机器将全部淘汰，试问应如何安排生产，使三年内收入最多？计划以一年为周期。

设x_1,x_2,x_3分别是第1、2、3年中用以生产产品P_1的机器数，y_1,y_2,y_3分别是第1、2、3年中用以生产产品P_2的机器数，则最大收入为：

$$\max z=50(x_1+x_2+x_3)+40(y_1+y_2+y_3)$$

约束条件：

$$x_1+y_1=1000$$

$$x_2+y_2=0.35x_1+0.6y_1$$

$$x_3+y_3=0.35x_2+0.6y_2 \qquad x_i,y_i>=0,i=1,2,3$$

算法设计策略——动态规划（Dynamic Programming）

2. 决策过程的分类

1）根据过程的时间变量是离散的还是连续的，分为*离散时间决策过程(discrete-time decision process)*，即多阶段决策过程和*连续时间决策过程(continuous-time decision process)*；

2）根据过程的演变是确定的还是随机的，分为*确定性决策过程(deterministic decision process)*和*随机性决策过程(stochastic decision process)*；

其中应用最广的是确定性、多阶段决策过程。

3. 动态规划模型的基本要素

一个多阶段决策过程最优化问题的动态规划模型通常包含以下要素：

（1）阶段：

简单说，就是把问题分成几个相互联系的有顺序的几个环节，这些环节即称为阶段。

*阶段(step)*是对整个过程的自然划分。通常根据时间顺序或空间特征来划分阶段，以便按阶段的次序求解优化问题。阶段变量一般用k=1,2,..,n表示。在例1中由A出发为k=1，由B_i(i=1,2)出发为k=2，依此下去从D_i(i=1,2,3)出发为k=4，共n=4个阶段。在例2中按照第一、二、三、四季度分为k=1,2,3,4，共4个阶段。

(2) 状态:

简单说，就是某一阶段的出发位置称为状态。通常一个阶段包含若干状态。

*状态(state)*表示每个阶段开始时过程所处的自然状况。是不可控因素。

描述状态的变量称*状态变量(state variable)*。用x_k表示第k阶段的状态变量，它可以是一个数或一个向量。

变量允许取值的范围称*允许状态集合(set of admissible states)*。用X_k表示第k阶段的允许状态集合。在例1中x_2可取B_1，B_2，$X_2=\{B_1,B_2\}$。

n个阶段的决策过程有n+1个状态变量，x_{n+1}表示x_n演变的结果，在例1中x_5取E。

根据过程演变的具体情况，状态变量可以是离散的或连续的。为了计算的方便有时将连续变量离散化；为了分析的方便有时又将离散变量视为连续的。

状态变量简称为*状态*。

算法设计策略——动态规划（Dynamic Programming）

需要特别指出，状态应该能够描述过程的特征并且具有*无后向性*。另外，通常还要求状态是直接或间接可以观测的。

无后向性，又称为马尔科夫性。是指系统从某个阶段后的发展，完全由本阶段所处的状态及其往后的决策决定，与系统以前的状态和决策无关。即当某阶段的状态给定时，这个阶段以后过程的演变与该阶段以前各阶段的状态无关，即每个状态都是过去历史的一个完整总结。

具有无后向性的多阶段决策过程，意味着系统过程的历史只能通过现阶段的状态去影响系统的未来，即当前状态就是过程往后发展的初始条件。

(3) 决策：

简单说就是从某阶段的一个状态演变到下一个阶段某状态的选择。

当一个阶段的状态确定后，可以作出各种选择从而演变到下一阶段的某个状态，这种选择手段称为*决策(decision)*，在最优控制问题中也称为*控制(control)*。

描述决策的变量称*决策变量(decision variable)*。用$u_k(x_k)$表示第k阶段处于状态x_k时的决策变量，它是x_k的函数。

变量允许取值的范围称*允许决策集合(set of admissible decisions)*。用$U_k(x_k)$表示了x_k的允许决策集合。在例1中$u_2(B_1)$可取C_1,C_2,C_3。

决策变量简称*决策*。

（4）策略：

简单说就是由开始到终点的全过程中，由每段决策组成的决策序列称为全过程策略，简称策略。

由初始状态x_1开始的全过程的策略记作$p_{1n}(x_1)$，即$p_{1n}(x_1)=\{u_1(x_1),u_2(x_2),\ldots,u_n(x_n)\}$。

由第k阶段开始到终止状态的过程称为后部子过程，亦称为k子过程。

由第k阶段的状态x_k开始到终止状态的后部子过程的策略记作$p_{kn}(x_k)$，即$p_{kn}(x_k)=\{u_k(x_k),u_{k+1}(x_{k+1}),\ldots,u_n(x_n)\}$。

算法设计策略——动态规划（Dynamic Programming）

类似地，由第k到第j阶段的子过程的策略记作$p_{kj}(x_k)=\{u_k(x_k),u_{k+1}(x_{k+1}),\ldots,u_j(x_j)\}$。

对于每一个阶段k的某一给定的状态x_k，可供选择的策略$p_{kj}(x_k)$有一定的范围，称为*允许策略集合(set of admissible policies)*。

$P_{1n}(x_1)$：全过程策略集合

$P_{kn}(x_k)$：后部子过程策略集合

$P_{1k}(x_1)$：前部子过程策略集合

$P_{kj}(x_k)$：k-j子过程策略集合

允许策略集合中达到最优效果的策略称为最优策略。

（5）状态转移方程：

简单说，前一阶段的终点就是后一阶段的起点，前一阶段的决策选择导出了后一阶段的状态，这种关系描述了由k阶段到k+1阶段状态的演变规律，称为状态转移方程。

在确定性过程中，一旦某阶段的状态和决策为已知，下阶段的状态便完全确定。用*状态转移方程(equation of state)*表示这种演变规律，写作：

$$x_{k+1} = T_k(x_k, u_k(x_k)), k = 1,2,...,n$$

在例1中状态转移方程为：$x_{k+1}=u_k(x_k)$

（6）指标函数和最优值函数

*指标函数(objective function)*是衡量过程优劣的数量指标，它是定义在全过程和所有后部子过程上的数量函数。亦即是关于策略的数量函数。

$V_{1n}(x_1, p_{1n}(x_1))$：表示从初始状态为x_1，采取策略为$p_{1n}(x_1)$时，全过程的指标函数值。一般地，从第k阶段的状态x_k到第n阶段的状态x_n决策过程（策略为$p_{kn}(x_k)$ ）的指标函数记作： $V_{kn}(x_k, p_{kn}(x_k))$， k=1, 2, . . . , n。

能够用动态规划解决的问题的指标函数应具有*可分离性，并满足递推关系。*即V_{kn}可表为$x_k, u_k, V_{k+1\ n}$ 的函数，记为：

$$V_{kn}(x_k, u_k, x_{k+1}, \ldots, x_{n+1}) = \varphi_k(x_k, u_k, V_{k+1n}(x_{k+1}, \ldots, x_{n+1}))$$

其中函数φ_k 是一个关于变量$V_{k+1\ n}$单调递增的函数。这一性质保证了*最优化原理(principle of optimality)*的成立，是动态规划的适用前提。

算法设计策略——动态规划（Dynamic Programming）

过程在第j阶段的阶段指标取决于状态x_j和决策u_j，用$v_j(x_j,u_j)$表示。阶段k到阶段n的决策指标函数值由v_j组成，$j=k,k+1,..n$。　　常见的形式有：

a. 阶段指标之和，即：$V_{kn}(x_k,u_k,...,x_{n+1})=\sum_{j=k}^{n}v_j(x_j,u_j)$

b.阶段指标之积，即：$V_{kn}(x_k,u_k,...,x_{n+1})=\prod_{j=k}^{n}v_j(x_j,u_j)$

c.阶段指标之最，即：$V_{kn}(x_k,u_k,...,x_{n+1})=\max_{k\le j\le n}(v_j(x_j,u_j))or\min_{k\le j\le n}(v_j(x_j,u_j))$

容易证明，这三个形式的指标函数都满足最优性原理。

（7）最优策略和最优轨线：

使指标函数V_{kn}达到最优值的策略称为*k后部子过程的最优策略*，记作$p_{kn}^{*}=\{u_k^{*},..,u_n^{*}\}$。

使指标函数V_{1n}达到最优值的策略称为全过程的最优策略，简称*最优策略(optimal policy)*，记作$p_{1n}^{*}=\{u_1^{*},...,u_n^{*}\}$

从初始状态$x_1(=x_1^{*})$出发，过程按照最优策略p_{1n}^{*}和状态转移方程演变所经历的状态序列$\{x_1^{*}, x_2^{*}, .., x_{n+1}^{*}\}$称*最优轨线(optimal trajectory)*。

算法设计策略——动态规划（Dynamic Programming）

总　结：

阶段$_1$	阶段$_2$	……	阶段$_n$	
状态集合$_1$	状态集合$_2$		状态集合$_n$	x_k, X_K
决策集合$_1$	决策集合$_2$		决策集合$_n$	$u_k(x_k)$, $U_k(X_K)$

状态转移方程：$x_{k+1} = T_k(x_k, u_k)$ 某阶段状态和决策已知，下一阶段的状态便确定。

策略、允许策略集合：$p_{kj}(x_k)=\{ u_k(x_k), u_{k+1}(x_{k+1}), \ldots, u_j(x_j) \}$

P_{1n}　P_{kn}　P_{kj}

算法设计策略——动态规划（Dynamic Programming）

指标函数(准则函数)：关于策略的函数，衡量策略（过程）的优劣。

定义在全过程上的指标函数相当于目标函数，一般记为 $V_{1n}(x_1, p_{1n})$，或简记为V_{1n}

定义在k子过程上的指标函数，记为$V_{kn}(x_k, p_{kn})$,简记为V_{kn}

指标函数在第k阶段一个阶段内的取值称为第k阶段的准则函数，记为$v_k(x_k, u_k)$

最优性原理中，指标函数具有（阶段）可分性，即

$$V_{k,n} = v_k(x_k, u_k) \oplus v_{k+1}(x_{k+1}, u_{k+1}) \oplus \cdots \oplus v_n(x_n, u_n),$$

其中 $\oplus$ 是满足单调性的某种运算，如加法或乘法等。

最优值函数：在x_k给定时，指标函数V_{kn}对p_{kn}的最优值记作$f_k(x_k)$，即

$$f_k(x_k) = \underset{p_{kn} \in P_{kn}(x_k)}{opt} V_{kn}(x_k, p_{kn}) \qquad (6) \quad f_1(x_1) = \underset{p_{1n} \in P_{1n}(x_1)}{opt} V_{1n}(x_1, p_{1n})$$

二、动态规划的基本定理和基本方程

1. 基本定理

对于初始状态$x_1 \in X_1$，策略$p_{1n}^* = \{u_1^*, ..u_n^*\}$是最优策略的充要条件是对于任意的$k$，$1<k<=n$，有

$$V_{1n}(x_1, p_{1n}^*) = \varphi\left(\underset{p_{1k-1} \in P_{1k-1}(x_1)}{opt} [V_{1k-1}(x_1, p_{1k-1})], \underset{p_{kn} \in P_{kn}(x_k)}{opt} [V_{kn}(x_k, p_{kn})] \right)$$

其中 $x_k = T_{k-1}(x_{k-1}, u_{k-1})$

即x_k 是由给定的初始状态x_1和子策略$p_{1\,k-1}$所确定的第k阶段的状态。

证明：略！

二、动态规划的基本定理和基本方程

2. 推论

若$p_{1n}^* \in P_{1n}(x_1)$是最优策略，则对于任意的$k,1<k<n$，它的子策略p_{kn}^*对于由x_1和$p_{1,k-1}^*$确定的以x_k^*为起点的第k到n后部子过程而言，也是最优策略。

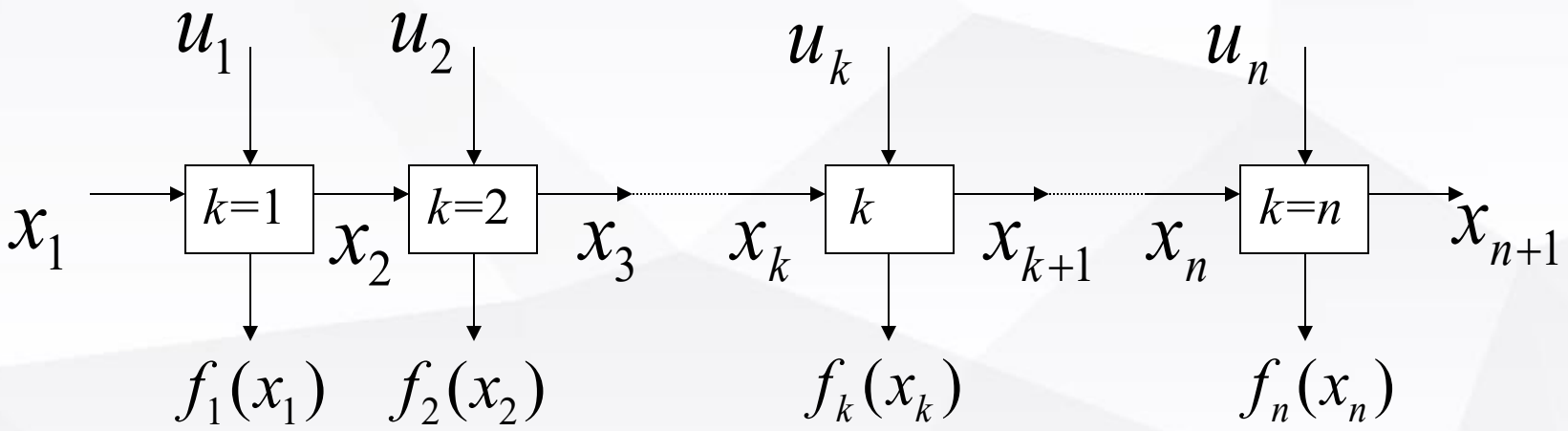
上述推论称为*最优化原理*，它给出了最优策略的必要条件，通常略述为：不论过去的状态和决策如何，对于前面的决策形成的当前的状态而言，余下的各个决策必定构成最优策略。即：最优策略的任一后部子策略都是最优的。

二、动态规划的基本定理和基本方程

3. 动态规划的基本方程

根据最优化原理，我们可以推出动态规划的基本方程

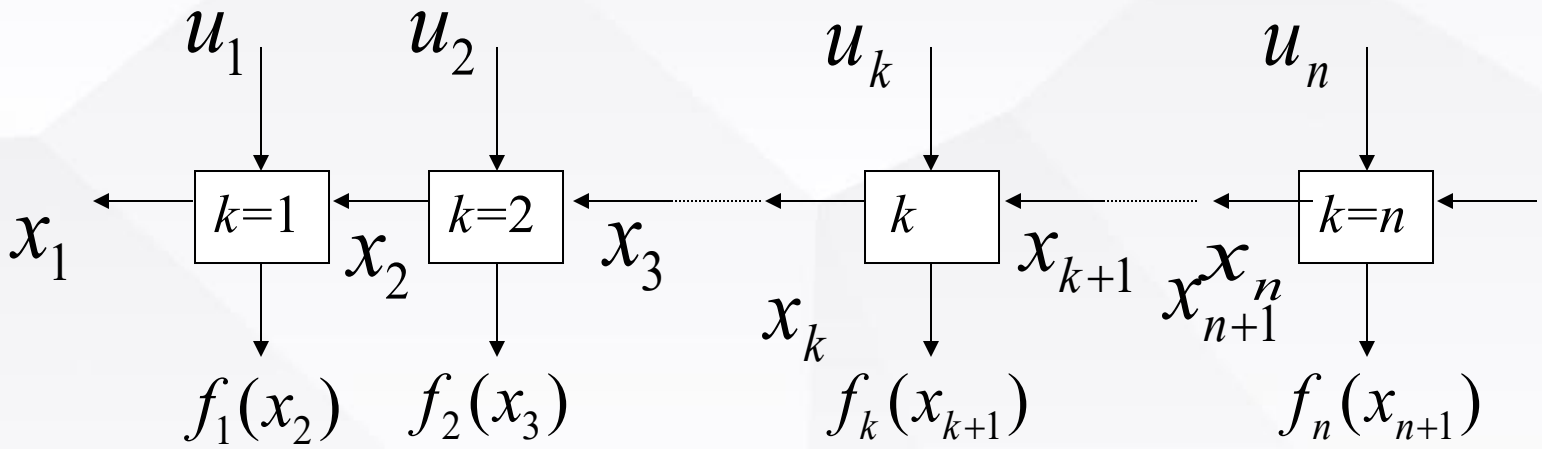
逆序递推式：



$f_k(x_k)$表示第k阶段初始状态为x_k时，k后部子过程的最优指标函数。

算法设计策略——动态规划（Dynamic Programming）

顺序递推式：



$f_k(x_{k+1})$表示第k阶段（结束）状态为x_{k+1}时，起始阶段到k阶段（可以称为k前部子过程）的最优指标函数 。

$$\begin{cases} f_k(x_{k+1}) = \min\limits_{u_k \in U_k} [v_k(x_{k+1}, u_k) + f_{k-1}(x_k)] \\ f_0(x_1) = 0 \end{cases}$$

$$x_k = G_k(x_{k+1}, u_k)$$

三、动态规划的适用条件

任何思想方法都有一定的局限性，超出了特定条件，它就失去了作用。同样，动态规划也并不是万能的。适用动态规划的问题必须满足***最优化原理、无后效性、重叠子问题***性质。

1. 最优化原理（最优子结构性质）

最优化原理可这样阐述：一个最优化策略具有这样的性质，不论过去状态和决策如何，对前面的决策所形成的状态而言，余下的诸决策必须构成最优策略。简而言之，一个最优化策略的子策略总是最优的，即问题的最优解包含了其子问题的最优解。一个问题满足最优化原理又称其具有*最优子结构性质*。

最优化原理是动态规划的基础，任何问题，如果失去了最优化原理的支持，就不可能用动态规划方法计算。动态规划的最优化原理在其指标函数的可分离性和单调性中得到体现。根据最优化原理导出的动态规划基本方程是解决一切动态规划问题的基本方法。

2. 无后向性

将各阶段按照一定的次序排列好之后，对于某个给定的阶段状态，它以前各阶段的状态无法直接影响它未来的决策，而只能通过当前的这个状态。换句话说，每个状态都是过去历史的一个完整总结。这就是*无后向性*，又称为*无后效性*。

如果用前面的记号来描述无后向性，就是：对于确定的x_k，无论$p_{1,k-1}$如何，最优子策略p_{kn}*是唯一确定的，这种性质称为无后向性。

***注意：*有些问题乍一看好像有后向性，但如果按照某种合理的方式重新划分阶段，就可以发现其本质上是无后向性的，所以关键是阶段的合理划分，这就是动态规划中的技巧了！**

3. 子问题的重叠性

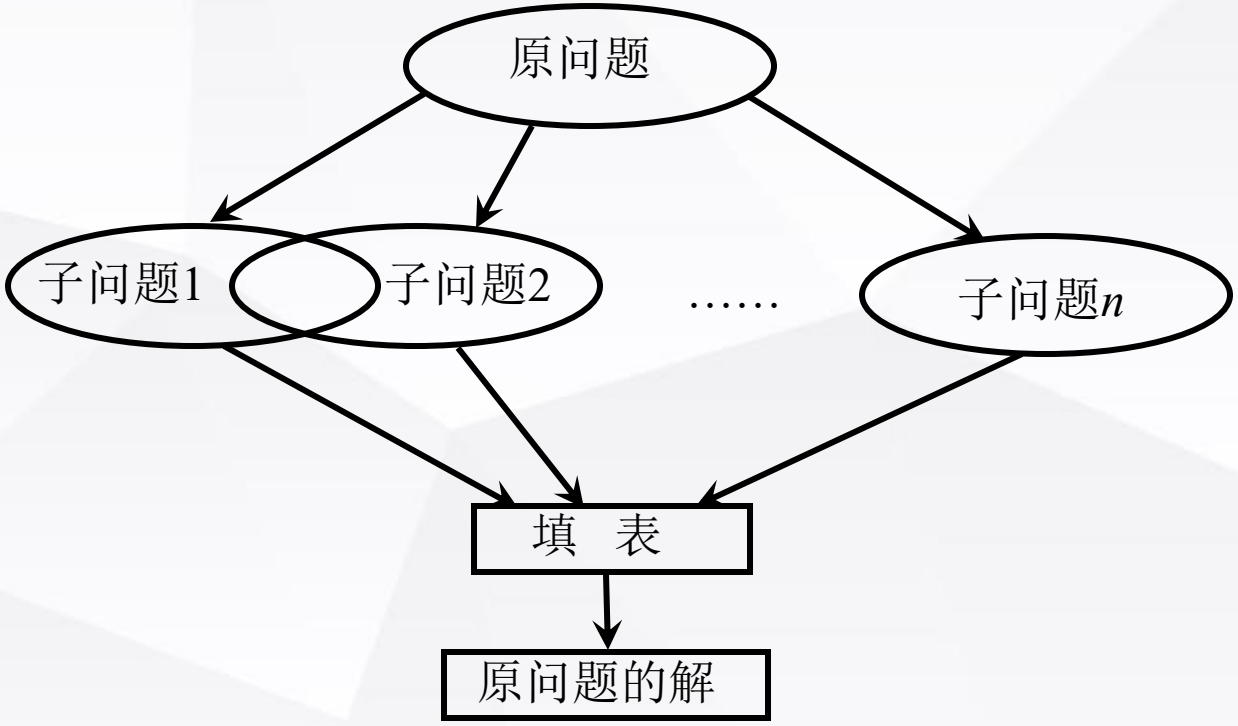
设原问题的规模为n，容易看出，当子问题树中的子问题总数是n的超多项式函数，而不同的子问题数只是n的多项式函数时，动态规划法显得特别有意义，此时动态规划法具有线性时间复杂性。所以，能够用动态规划解决的问题还有一个显著特征：*子问题的重叠性*。这个性质并不是动态规划适用的必要条件，但是如果该性质无法满足，动态规划算法同其他算法相比就不具备优势。

我们看到，动态规划将原来具有指数级复杂度的搜索算法改进成了具有多项式时间的算法。其中的关键在于解决冗余，这是动态规划算法的根本目的。动态规划实质上是一种以空间换时间的技术，它在实现的过程中，不得不存储产生过程中的各种状态，所以它的空间复杂度要大于其它的算法。

四、动态规划的基本思想

具有明显的阶段划分和状态转移方程的动态规划称为*标准动态规划*，这种标准动态规划是在研究多阶段决策问题时推导出来的，具有严格的数学形式，适合用于理论上的分析。

在实际应用中，许多问题的阶段划分并不明显，这时如果刻意地划分阶段反而麻烦。一般来说，只要该问题可以划分成规模更小的子问题，并且原问题的最优解中包含了子问题的最优解（即满足最优子化原理），则可以考虑用动态规划解决。



算法设计策略——动态规划（Dynamic Programming）

四、动态规划的基本思想

■ 动态规划法与分治法类似，它们都是将问题实例归纳为更小的、相似的子问题，并通过求解子问题产生一个问题的解。但是，对于动态规划策略，采用了解决冗余的思想。

■分治法中的各个子问题是独立的（即不包含公共的子问题），因此一旦递归地求出各子问题的解后，便可自下而上地将子问题的解合并成问题的解。如果有重复子问题，效率会很低。

■ 动态规划法与贪心策略类似，它们都是解决最优化问题，都是通过求解局部子问题的解达到全局最优解。

■贪心策略当前选择可能要依赖已经作出的所有选择，但不依赖于有待于做出的选择和子问题。因此贪心法自顶向下，一步一步地作出贪心选择，即是从局部最优出发的；而动态规划的决策不依赖于过去的，只与当前状态有关，是从全局最优出发的。

四、动态规划的基本思想

动态规划法所针对的问题有一个显著的特征，即它所对应的子问题树中的子问题呈现大量的重复。另外能刻画出整体最优和局部最优的关系(方程)，即较小子问题的最优解与较大子问题的最优解之间存在的数值关系。

建立一个标准的动态规划算法的基本过程是：

1．划分阶段：按照问题的时间或空间特征，把问题划分为若干阶段。注意：这若干个阶段一定要是有序或可排序的（满足无后向性），否则就无法用动态规划求解。

2．选择状态：将问题发展到各个阶段时所处的各种客观情况用不同的状态表示出来。当然，状态的选择必须满足无后向性。

算法设计策略——动态规划（Dynamic Programming）

3．确定决策并写出状态转移方程：这本来是两个步骤，但是放在了一起，因为决策和状态转移有着天然的联系。状态转移方程就是根据上一阶段的状态和决策来导出本阶段的状态。因此，确定了决策，状态转移方程就可以写出来了。但是通常反过来做，根据两个相邻阶段的各状态之间的关系来确定决策。

即确定 u_k 和 $x_{k+1}= T_k(x_k, u_k)$。 $v_k(x_k,u_k)$

4．写出动态规划方程（包括边界条件）：只要阶段、状态、决策、状态转移确定了，写出动态规划方程是很容易的，即写出$f_k(x_k)$

$$\begin{cases} f_k(x_k)=\underset{u_k\in U_k(x_k)}{opt}\{\varphi(v_k(x_k,u_k),f_{k+1}(x_{k+1}))\}\ ,\ x_{k+1}=T_k(x_k,u_k)\ ,\ k=1,2,\cdots,n \\ f_{n+1}(x_{n+1})=\delta(x_{n+1}) \end{cases}$$

$$opt\{V_{1n}\}=\underset{x_1\in X_1}{opt}\{f(x_1)\}$$

算法设计策略——动态规划（Dynamic Programming）

5. 求解动态规划方程（以自底向上递推或自顶向下递归的方式），计算出最优值。

6. 根据计算最优值得到的（决策）信息，构造一个最优解。

动态规划的难点在于理论上的设计，一旦设计完成，实现起来非常简单，也就是根据动态规划方程进行求解（递归，或者递推）。标准动态规划递推求解算法框架如下：

算法设计策略——动态规划（Dynamic Programming）

```
Dynamic（P)
{ // 递推求解标准动态规划方程，即动态规划策略设计的算法
    对于f_{n+1}(x_{n+1})初始化；//边界条件
    for( k=n;k<=1;k--)     //n个阶段
        for( 每一个x_k ∈X_k)
            for( 每一个u_k ∈U_k(x_k) )
              { f_k(x_k) = 一个极值;        //∞ ,-∞
                x_{k+1} = T_k(x_k,u_k);     //状态转移方程
                 t = φ( v_k(x_k,u_k),f_{k+1}(x_{k+1}) ); //规划方程
                 if( t比f_k(x_k)更优 ) f_k(x_k) = t;  //计算f_k(x_k)最优值
              }
              t = 一个极值;
     for( 每一个x_1 ∈X_1)
      if( t比f_1(x_1)更优) f_1(x_1)=t;  //计算f_1(x_1)最优值
  输出t;
}
```

算法设计策略——动态规划（Dynamic Programming）

但在实际应用时，很少有标准的动态规划方程，因此一般不是严格按照上面的来做，而是按照下面步骤来进行设计算法：——非标准动态规划算法设计

（1）分析最优解的性质，并刻画其结构特征。

（2）递归定义最优值；（写出动态规划方程）

（3）求解动态规划方程（以自底向上递推或自顶向下递归的方式），计算出最优值；

（4）根据计算最优值得到的信息，构造一个最优解；

注意，如果只是求最优值，则可以省略（4）；如果要求出解轨迹，即如何达到最优值的解，则需要在（3）中记录一些信息，然后在（4）中构造出最优解。

算法设计策略——动态规划（Dynamic Programming）

动态规划方程是一个递归式，求解可以采用自顶向下的递归式求解，这时，需要“备忘录”。也可以采用自底向上的方式递推分步计算，即先求较小子问题的最优解，然后从它们的最优解得到较大子问题的最优解，直至最终求得原问题的最优解。

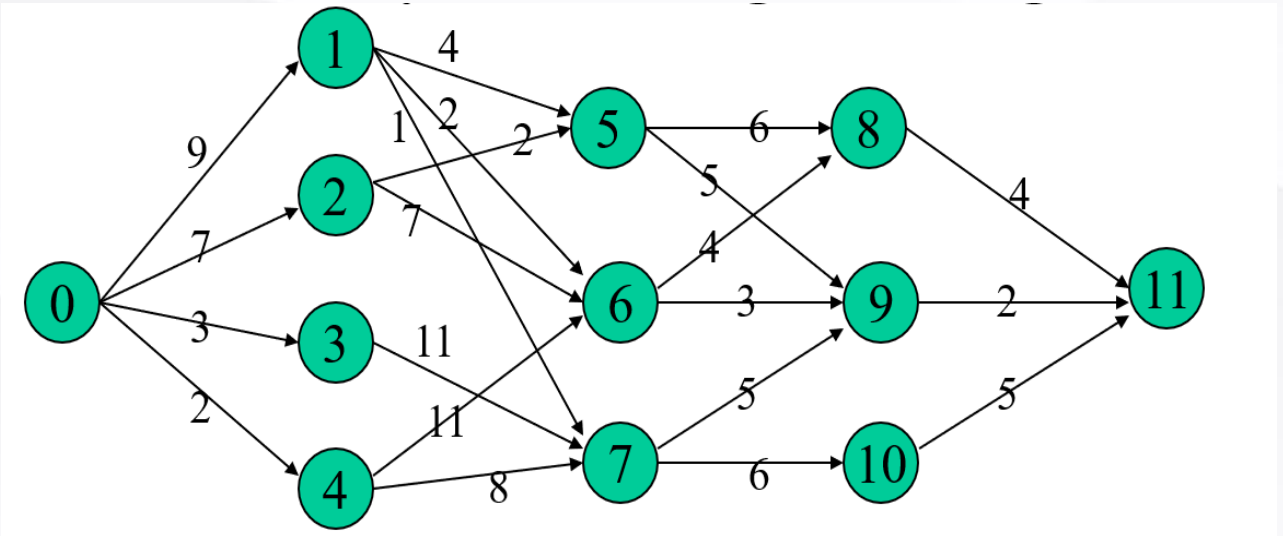
由于各种问题的性质不同，判定最优解的方法也不相同，因此，对于不同的问题，动态规划的方法也不相同。不存在万能的动态规划方法。

六、举例

例1：多段图的最短路径问题

多段图G=（V，E）是一个加权的有向图，它具有如下的特性：图中的结点被划分为k≥2个互不相交的子集V_i，$1 \leqslant i \leqslant k$。其中$V_1$和$V_k$分别只有一个结点（分别称为源点s和汇点t），对于所有边$<u,v> \in E$，多段图要求若$u \in V_i$则$v \in V_{i+1}(1<i \leqslant k)$，边上的权值为w(u,v)。从s到t的路径长度是这条路径上边的权值之和。多段图问题是求从s到t的一条最短路径。多段图问题是一种*特殊的有向无环图的最短路径*问题。

从s到t的每一条路径都是一个可行解，每条路径上所有边的权值之和称为路径长度，这是问题的目标函数值，用此函数值来衡量一个决策序列（策略）的优劣。



（1）多段图的最优子结构特性

假设$(s,v_2,v_3,\cdots v_{k-1},t)$是一条从s到t的最短路径，并假定在初始状态已经做出了达到v_2的决策，即v_2是从初始状态s，由初始决策所到达的状态。如果将v_2看成原问题的一个子问题的初始状态，求解此子问题就是寻找一条从v_2到t的最短路径。如果$(s,v_2,v_3,\cdots,v_{k-1},t)$是一条从s到t的最短路径，那么$(v_2,v_3,\cdots,v_{k-1},t)$必定是一条$v_2$到t的最短路径。

证明：因为，若不然，另有$(v_2,q_3,\cdots q_{k-1},t)$是$v_2$到t的最短路径，那么路径$(s,v_2,q_3,\cdots,q_{k-1},t)$显然比$(s,v_2,v_3,\cdots,v_{k-1},t)$更短，与假设矛盾。

（2）向前递推关系式

动态规划的每一步的决策依赖于子问题的解。对于多段图问题，一个阶段的决策与后面所要求解的子问题相关，所以不能在某个阶段直接做出最优决策。

但是由于多段图具有最优子结构特性，启发我们从最后阶段开始，采用逐步向前递推的方式，由子问题的最优解来计算出原问题的最优解。

算法设计策略——动态规划（Dynamic Programming）

假设i是阶段编号，j是第i阶段的一个状态编号，cost(i,j)是从第i阶段状态j到t的最短路径的长度。于是有：

cost(k,t)=0　　//第k个阶段的t状态到t的最短距离

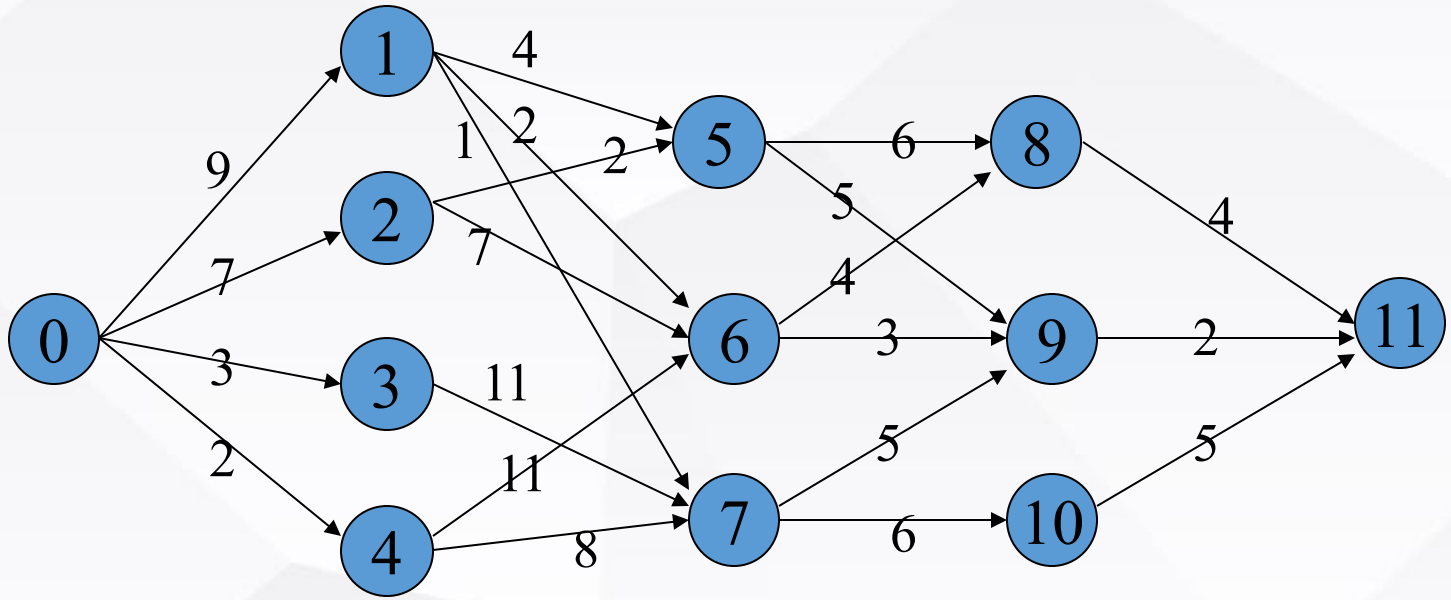
$$cost(i,j)=\min_{\substack{j\in V_i,\ p\in V_{i+1}\\ <j,p>\in E}}\{c(j,p)+cost(i+1,p)\}\qquad 0\le i\le k-2$$

//为了计算cost(i,j)，必须先计算从j的所有后继结点p到t的最短路径的长度，即cost(i+1,p)

一般地，为了计算cost(i,j)，必须先计算从j的所有后继结点p到t的最短路径长度，即先计算cost(i+1,p)的值，这是子问题的最优解。c(j,p)+cost(i+1,p)是从第i阶段结点j，经过第i+1阶段结点p到汇点t的最短路径的长度。cost(i,j)是这些路径中的最短路径长度。最终得到的cost(1,s)就是多段图问题的最优解值。

算法设计策略——动态规划（Dynamic Programming）

例如，下面的5段图



cost(5,11)=0

cost(4,10)=min{ <10,11>:5+ cost(5,11):0 }=5　　$u_4(10)=11$

cost(4,9)=min{ <9,11>:2+ cost(5,11):0 }=2　　$u_4(9)==11$

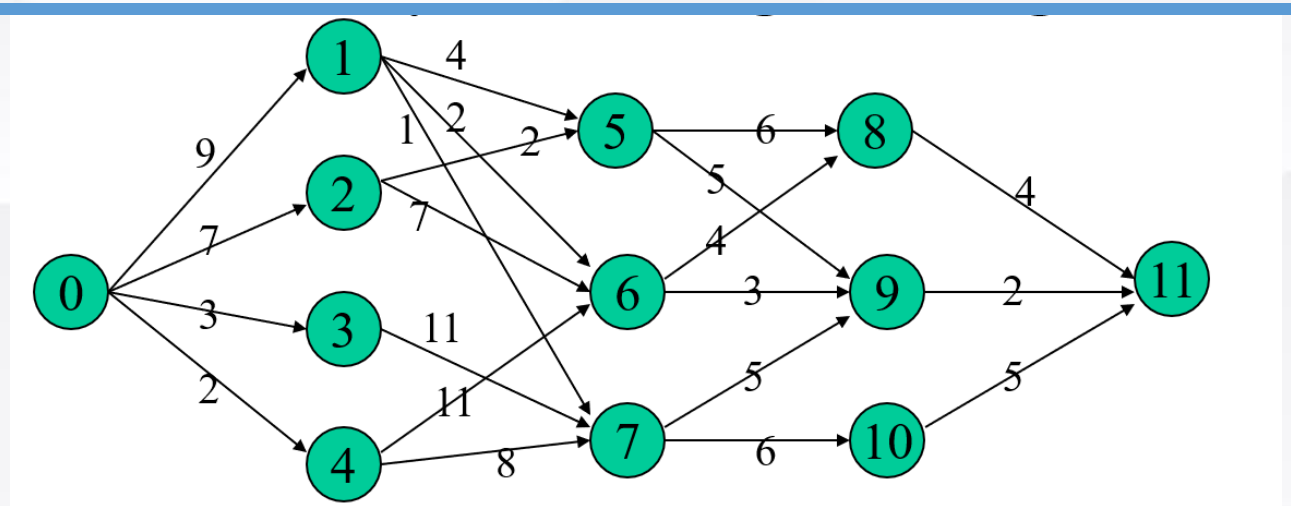
cost(4,8)=min{<8,11>:4+ cost(5,11):0 }=4　　$u_4(8)==11$

cost(3,7)=min{ <7,9>:5+cost(4,9):2 , <7,10>:6+cost(4,10):5 }=7　　$u_3(7)==9$

cost(3,6)=min{ <6,8>:4+cost(4,8):4 , <6,9>:3+cost(4,9):2 }=5　　$u_3(6)==9$

cost(3,5)=min{ <5,8>:6+cost(4,8):4 , <5,9>:5+cost(4,9):2} =7　　$u_3(5)==9$

算法设计策略——动态规划（Dynamic Programming）



cost(2,1)=min{ <1,5>:4+cost(3,5):7 , <1,6>:2+cost(3,6):5,
<1,7>:1+cost(3,7):7 }= 7

$u_2(1)==6$

cost(2,2)=min{<2,5>:2+cost(3,5):7 , <2,6>:7+cost(3,6):5 }= 9

$u_2(2)==5$

cost(2,3)=min{ <3,7>:11+cost(3,7):7 }= 18

$u_2(3)==7$

cost(2,4)=min{<4,6>:11+cost(3,6):5 , <4,7>:8+cost(3,7):7 }= 15

$u_2(4)==7$

cost(1,0)=min{ <0,1>:9+cost(2,1):7 , <0,2>:7+cost(2,2):9,
<0,3>:3+cost(2,3):18 , <0,4>:2+cost(2,4):15 } = 16

$u_1(0)==1$

$u_1(0)==2$

最短路径长度为16。

但是为了求出是那条最短路径，必须在求解的过程中记录信息，即每次的决策！

例如用d(i,j)来记录从第i阶段结点j到t的最短路径上该结点的下一个结点编号。即决策选择的结点（子问题的最优解）。

d(4,10)=d(4,9)=d(4,8)=11

d(3,7)=d(3,6)=d(3,5)=9

d(2,4)=d(2,3)=7,　d(2,2)=5,　d(2,1)=6

d(1,0)=1或2

0→1 →6 →9 →11　　0→2 →5 →9 →11

（3）重叠子问题

前面分析我们看到，存在重叠子问题现象。例如，cost(3,5),cost(3,6),cost(3,7)的计算都用到cost(4,9)的值。如果保存cost(4,9)的值，就可以避免重复计算它的值。

（4）多段图的动态规划算法

由我们前面的递推公式知道，要求保证在计算某个结点j到t的最短路径长度时，结点j的所有后继结点p到t的最短路径长度已经由计算得到。所以需要对图G的结点按阶段顺序从0-n-1进行编号，源点s编号为0，汇点t编号为n-1。向前递推计算按照结点编号从大到小的次序进行：

先计算cost[n-1]=0，再计算cost[n-2]，……，最后计算得到cost[0]，这就是最短路径长度。另用一维数组p保存对应于cost[0]的最短路径上的结点，即最优解。

算法设计策略——动态规划（Dynamic Programming）

算法如下：

——标准的动态规划算法

```
Fpath(G,k,p)
{ float cost[n]; int d[n];
  cost[n-1]=0; d[n-1]=-1;   //设置向前递推的初值
  for(j=n-2,j>=0;j--)        //按n-2,…,0的次序计算cost和d
    { length=MAX;
      for(r=j+1;r<=n-1;r++)   //按递推公式计算最小值cost[j]
        {   if(边<j,r>存在&&(<j,r>权+cost[r]<length))
              { length= <j,r>权+cost[r];   q=r;   }
        }
      cost[j]=length; d[j]=q;        //q是j在最短路径上的后继结点
    }
  p[0]=0; p[k-1]=n-1; c=cost[0];
  for(j=1;j<=k-2;j++)
    p[j]=d[p[j-1]];          //p[i]是最短路径上第i阶段的结点
}
```

算法设计策略——动态规划（Dynamic Programming）

我们前面分析时给出的递推公式，和下面的公式是一样的：

$$F_k(u)=\min_{u\in S_k,v\in S_{k+1}\cap\delta(u)}\{w(u,v)+F_{k+1}(v)\}$$

边界条件：　$F_5(E)=0$

（5）向后递推关系式

k段图问题也可以向后递推求解，其递推公式如下：

Bcost(1,s)=0

Bcost(i,j)= $\min_{p\in V_{i-1},j\in V_i,<p,j>\in E}\{c(p,j)+Bcost(i-1,p)\}$　$1<i\leq k$

向后递推的动态规划算法请自己写出！

例2：生产计划问题

问题描述：

工厂生产某种产品，每单位(千件)的成本为1(千元)，每次开工的固定成本为3(千元)，工厂每季度的最大生产能力为6(千件)。经调查，市场对该产品的需求量第一、二、三、四季度分别为 2，3，2，4(千件)。如果工厂在第一、二季度将全年的需求都生产出来，自然可以降低成本(少付固定成本费)，但是对于第三、四季度才能上市的产品需付存储费，每季每千件的存储费为0.5(千元)。还规定年初和年末这种产品均无库存。试制订一个生产计划，即安排每个季度的产量，使一年的总费用(生产成本和存储费)最少。

算法设计策略——动态规划（Dynamic Programming）

这是一个明显的多阶段问题，按照计划时间自然划分阶段，状态定义为每阶段开始时的存储量x_k，决策为每个阶段的产量u_k，记每个阶段的需求量（已知）为d_k，则状态转移方程为：

$$x_{k+1} = x_k + u_k - d_k, x_k \geq 0, k = 1, 2, \cdots, n$$

设每个阶段开工固定成本费用为a，生产单位数量产品的成本为b，每阶段单位数量产品的存储费用为 c ，阶段指标为阶段的生产成本费用和存储费用之和，即：

$$v_k(x_k, u_k) = cx_k + \begin{cases} a + bu_k, u_k > 0 \\ 0 \qquad\quad , u_k = 0 \end{cases}$$

算法设计策略——动态规划（Dynamic Programming）

指标函数V_{kn}为v_k之和，最优值函数$f_k(x_k)$为从第k阶段的状态x_k出发到过程终结的最小费用，满足：

$$f_k(x_k)=\min_{u_k\in U_k}[v_k(x_k,u_k)+f_{k+1}(x_{k+1})],k=n,\cdots,1$$

其中允许决策集合U_k由每阶段的最大生产能力决定，设过程终结时允许存储量为x^0_{n+1}，则终端条件为：

$$f_{n+1}(x^0_{n+1})=0$$

将以上各式代入到标准动态规划的框架中，就可以求得最优解。

算法略

例3：资源分配问题

将n个资源分配给r个项目，已知如果把j个资源分配给第i个项目，可以收益N(i,j)，0≤j≤n，1≤i≤r。求总收益最大的资源分配方案。

总收益 $$G=\sum_{i=1}^{r}N(i,j)$$

j是第i个项目分配的资源数目，$$\sum_{i=1}^{r}j_i=n$$

问题是：把n划分成r份，每份投入到相应的项目上，求使G达到最大的划分。

算法设计策略——动态规划（Dynamic Programming）

这样的问题，看似阶段性不明显，但是仔细分析发现，可以用一个多段图描述该问题。多段图共分为$r+1$阶段：

第1阶段是初始阶段，只包含一个初始状态s，表示分配没开始；

第$r+1$阶段是结束阶段，也只有一个结束状态t，表示分配完成；

除此之外的$r-1$个中间阶段，每个阶段包含$n+1$个状态，状态表示为$V(i,j)$，其中i是指第几阶段，j表示几个资源，$j=0,1,\ldots n$ $V(i,j)$状态的含义是已将j个资源分配给了前$i-1$项目，其中$s = V(1,0)$，$t = V(r+1,n)$。

图中的边具有$<V(i,j),V(i+1,k)>$ $(0 \le j \le k \le n,\ 1 \le i \le r)$的形式表示**第$i$个项目分配了$k-j$个资源**，边上的权值$N(i,k-j)$是第$i$个项目分配了$k-j$个资源的收益。

于是知道：

第2阶段的状态为$V(2,j)$，$j=0,1,2\ldots n$，表示j个资源分配给了前1个项目，边$<V(1,0),V(2,j)>$就表示第 1 个项目分配了j个资源，边上的权值$N(1,j)$表示第 1 个项目分配了j个资源的收益。

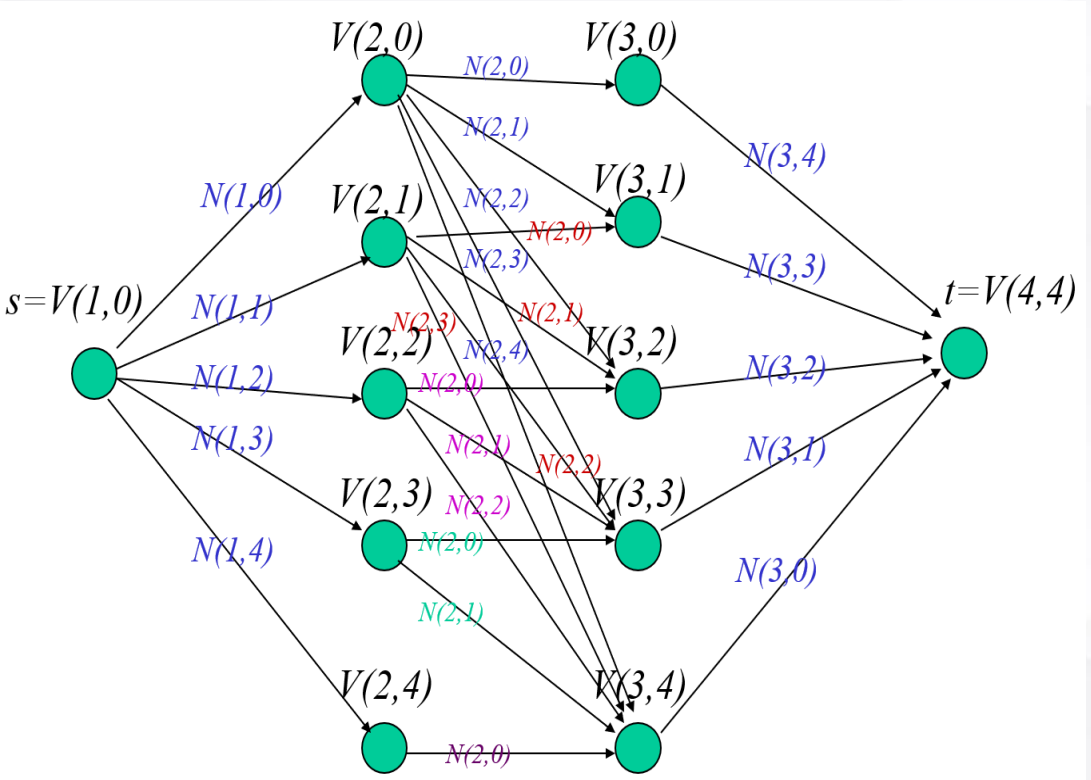
第3阶段的状态为$V(3,k)$，$k=0,1,2\ldots n$，表示k个资源分配给了前2个项目，边$<V(2,j),V(3,k)>$就表示第 2 个项目分配了$k-j$个资源, 边上的权值$N(2,k-j)$表示第2个项目分配了$k-j$个资源的收益。

……依此类推

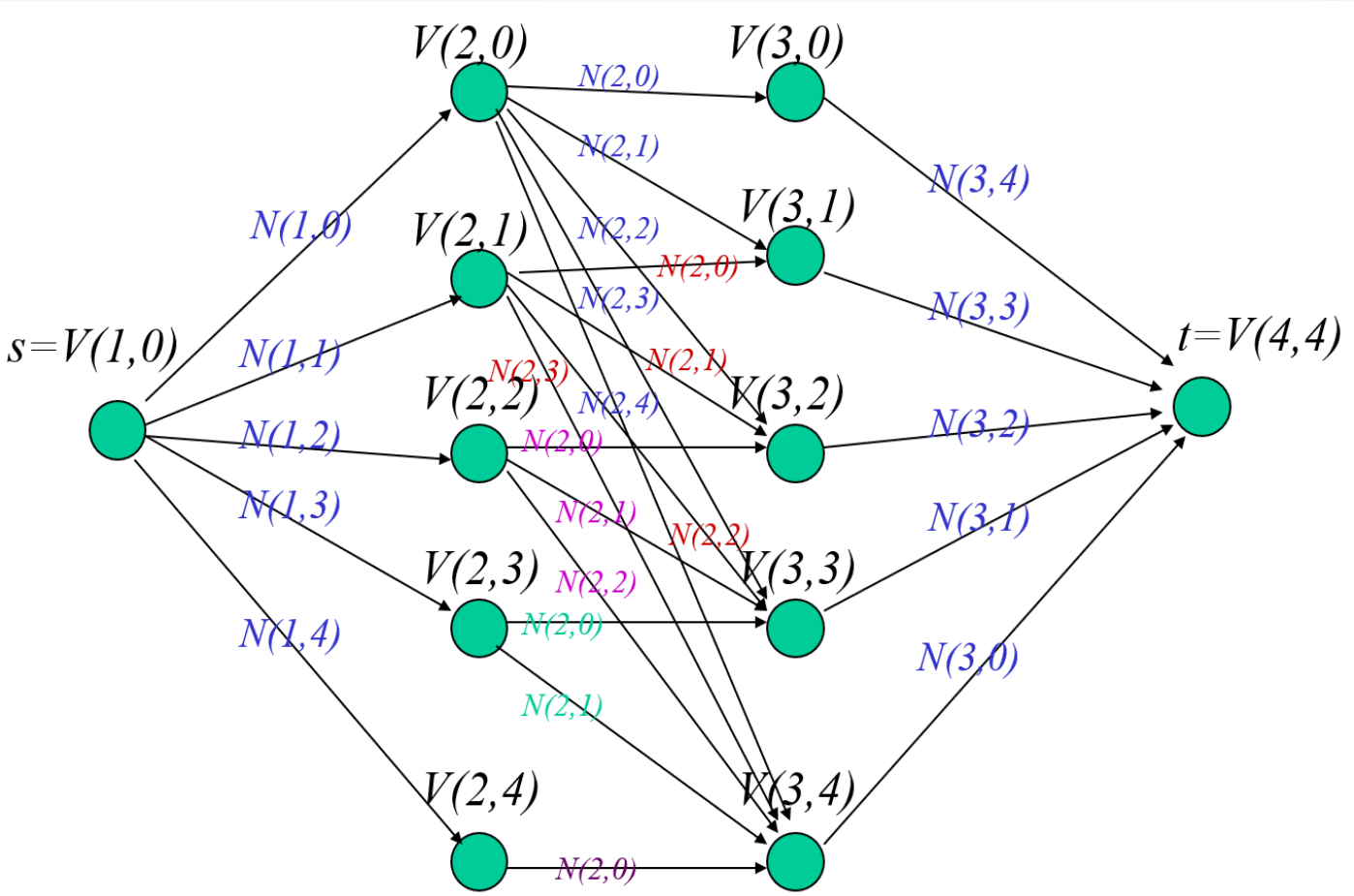
算法设计策略——动态规划（Dynamic Programming）

例如，将4个资源分配给3个项目。可以用一个4(即3+1)阶段图表示。*S=V(1,0)*为初始状态，表示尚未分配任何资源。*t=V(4,4)*是结束状态。

中间 *2*个阶段的形成如下：第 *2*阶段，具有 *5*个状态，分别表示为 *V(2,0), V(2,1), V(2,2), V(2,3), V(2,4)*,分别表示*0*个资源分配给前 *1* 个项目、 *1* 个资源分配给前 *1* 个项目、 *2* 个资源分配给前 *1* 个项目、 *3*个资源分配给前 *1* 个项目、 *4* 个资源分配给前 *1* 个项目。有5条边：*<V(1,0),V(2,0)>*, *<V(1,0),V(2,1)>*， *<V(1,0),V(2,2)>*， *<V(1,0),V(2,3)>*, *<V(1,0),V(2,4)>*，表示第*1*个项目分配了*0*个资源、第*1*个项目分配了*1*个资源、第*1*个项目分配了*2*个资源、第*1*个项目分配了*3*个资源、第*1*个项目分配了*4*个资源;边上的权值*N(1,0), N(1,1), N(1,2), N(1,3), N(1,4)*,分别表示各个分配的收益。

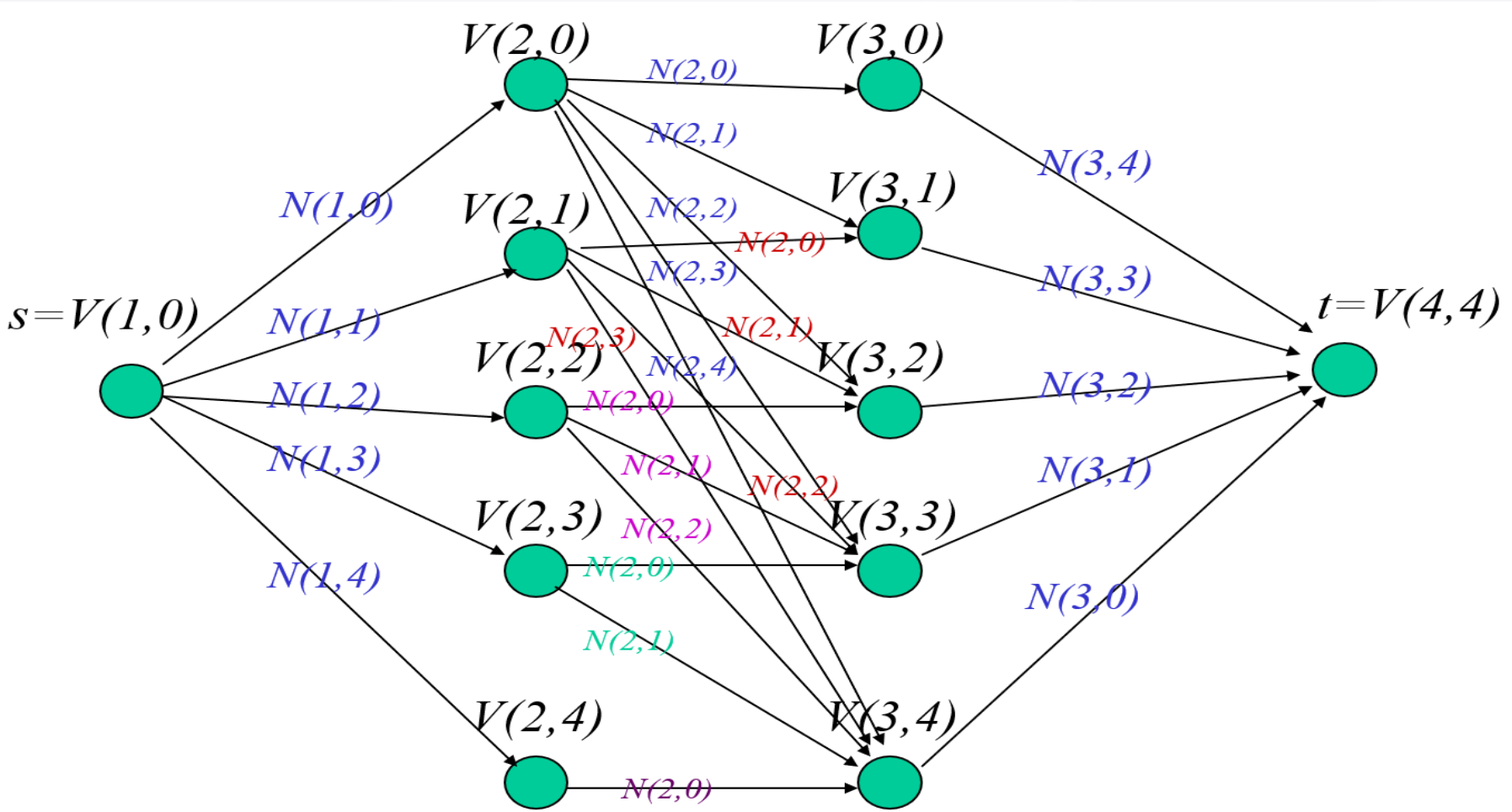


算法设计



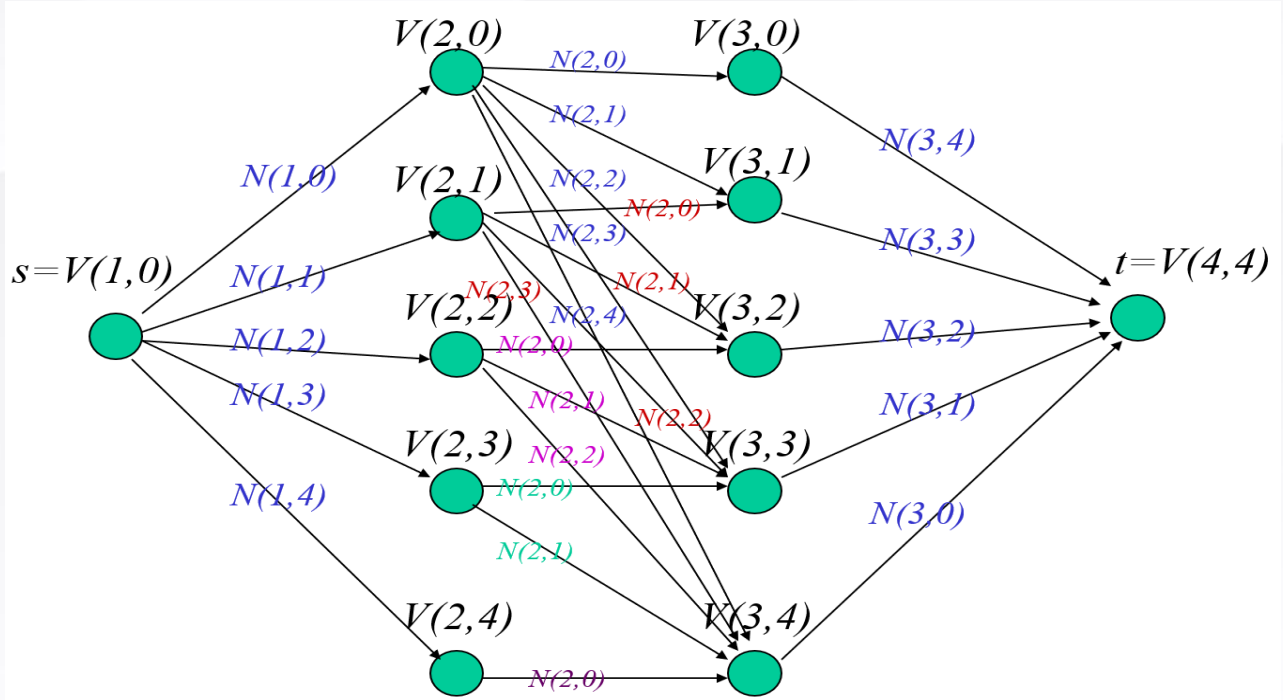
第 3阶段，具有 5个状态，分别表示为V(3,0), V(3,1), V(3,2), V(3,3), V(3,4),分别表示0个资源分配给前2个项目、 1 个资源分配给前2个项目、 2 个资源分配给前2个项目、 3 个资源分配给前2个项目、 4 个资源分配给前2个项目。有15条边：<V(2,0),V(3,0)>， <V(2,0),V(3,1)>， <V(2,0),V(3,2)>， <V(2,0),V(3,3)>， <V(2,0),V(3,4)>， <V(2,1),V(3,1)>，<V(2,1),V(3,2)>， <V(2,1),V(3,3)>， <V(2,1),V(3,4)>， <V(2,2),V(3,2)>， <V(2,2),V(3,3)>， <V(2,2),V(3,4)>，<V(2,3),V(3,3)>， <V(2,3),V(3,4)>， <V(2,4),V(3,4)>，表示第2个项目分配了0个资源、第2个项目分配了1个资源、第2个项目分配了2个资源、第2个项目分配了3个资源、第2个项目分配了4个资源；边上的权值N(2,0), N(2,1), N(2,2), N(2,3), N(2,4), N(2,0), N(2,1)， N(2,2), N(2,3), N(2,0), N(2,1), N(2,2), N(2,0), N(2,1), N(2,0)，分别表示各个分配的收益。

算法设计策略



第 3 阶段状态到结束状态的边有 5 条：<V(3,0),V(4,4)>， <V(3,1),V(4,4)>，<V(3,2),V(4,4)>， <V(3,3),V(4,4)>， <V(3,4),V(4,4)>， 表示第3个项目分配了4个资源、第3个项目分配了3个资源、第3个项目分配了2个资源、第3个项目分配了1个资源、第3个项目分配了0个资源;边上的权值N(3,4), N(3,3), N(3,2), N(3,1), N(3,0), 分别表示各个分配的收益。

算法设计策略——动态规划（Dynamic Programming）



从*s*到*t*的一条路径代表为*3*个项目分配的资源总数不超过*4*个的一种分配方案。

从*s*出发有 *5*条边，边上的权值表示将*j*个资源分配给第 *1*个项目的收益。不同的决策使分配进入第 *2*阶段的不同状态。例如*V(2,2)*表示已经将 *2*个资源分配给了第 *1*个项目。状态*V(3,2)*表示已经将*2*个资源分配给了第 *1*、 *2*个项目，此时还剩 *2*个资源分配给第 *3*个项目。

算法设计策略——动态规划（Dynamic Programming）

现假设将有限资源分配给若干个工程的问题，设资源总数是a，工程个数为r，给每项工程分配的资源数目不同，获得的利润也不相同，给定各工程的资源利润表和资源总数，应当如何分配资源，才能获得最大的利润呢？

即要求给出a一个拆分：

$x_1,x_2,\ldots,x_r$，　$0 \le x_i \le a$，且 $\sum_{i=1}^{r} x_i = a$，使得利润最大；

即：

$$max\{ G_1(x_1)+G_2(x_2)+\ldots\ldots+G_r(x_r) \}$$

其中 $G_i(x_i)$ 是把资源数x_i分配给第i项工程能获得的利润，$G_i(x)$ 为给定的已知函数，若 $G_i(x)$ 是x的线性函数，则就是一般的线性规划（LP，即 Linear Programming Problem）问题；对于线性规划问题的求解，数学上有自己的方法。

算法设计策略——动态规划（Dynamic Programming）

按照最优化问题的描述方法，该问题就可以描述为：

限制条件 $\begin{cases} 0 \le x_i \le a \\ \sum_{i=1}^{r} x_i = a \end{cases}$

优化函数 $\max\limits_{1 \le i \le r} \sum_{i}^{r} G_i(x_i)$

若上式中 $G_i(x)$ 是离散的（非线性函数），则可用动态规划求最佳分配。资源总数可以是投资总额，工程项目或产品种类。故资源分配问题是许多实际问题的抽象。

第一阶段，分别把 $x=0,1,\cdots,a$ 个资源分配给第一个工程，确定第一个工程在各种不同数目资源下能够得到的最大利润。

第二阶段，分别把 $x=0,1,\cdots,a$ 个资源分配给第一、二个工程，确定在各种不同数目资源下，这两个工程能够得到的最大利润，以及在此利润下，第二个工程的最优资源数目。

依次类推，在第r个阶段，分别把 $x=0,1,\cdots a$ 个资源分配给所有r个工程，确定能够得到的最大利润，以及在此利润下，第r个工程的最优分配资源数目。

算法设计策略——动态规划（Dynamic Programming）

由于把a个资源全部投入给所有r个工程（可能是投入到部分工程），不一定能够得到最大利润。

因此必须在各个阶段里，对不同的资源分配额计算能够得到得最大利润，然后取其中的最大者作为每个阶段能够取得的最大利润。再取每个阶段的最大利润中的最大者，以及在该利润下的分配方案，即为整个资源分配的最优决策。

为此，设$f_i(x)$表示把x个资源分配给前i个工程时所得到的最大利润，$d_i(x)$表示使$f_i(x)$最大时，分配给第i个工程的资源数。于是，在第一阶段，只把x个资源分配给第1个工程，有：

$$\begin{cases} f_1(x)=G_1(x) \\ d_1(x)=x \end{cases} \qquad 0\le x\le a$$

在第二阶段，只把x个资源分配给前面的两个工程，有：

$$\begin{cases} f_2(x)=max\{ G_2(z)+f_1(x-z) \} \\ d_2(x)=使f_2(x)达最大的\ z \end{cases} \quad 0\le x\le a \quad 0\le z\le x$$

一般地，在第i阶段，把x个资源分配给前面的i个工程，有：

$$\begin{cases} f_i(x)=max\{ G_i(z)+f_{i-1}(x-z) \} \\ d_i(x)=使f_i(x)达最大的\ z \end{cases} \quad 0\le x\le a \quad 0\le z\le x$$

令第i阶段的最大利润为g_i，则： $g_i = max\{ f_i(1), f_i(2),..., f_i(a) \}$

即 g_i是第 i 阶段分配不同资源数目时的最大利润；

设q_i是该阶段利润达到最大时分配给前面i个工程的资源数，即分配了多少个资源给前 i 个项目时，第 i 阶段利润达到最大。 则： q_i = 使$f_i(x)$达最大的x

相应的，$d_i(x)$ 是记录了最大$f_i(x)$时 分配给第i个项目的资源数；

因此，在每个阶段结束，都会得到：

g_i，q_i，$d_i(x)$

在第n阶段结束后，令全局的最大利润为$optg$，则：

$optg = max\{ g_1, g_2, \ldots, g_n \}$

然后可以确定，在第几个阶段达到了最大利润。假设g_k最大，则说明资源只分配给了前k个工程项目，即：　k = 使g_i最大的i

从而也可知，分配给前面k个工程项目的总资源数为：

$optx$ = 与最大g_i相对应的q_i

算法设计策略——动态规划（Dynamic Programming）

也就可以知道，分配给第k个工程的资源份额为：$optq_k = d_k(optx)$

进而知道，分配给其余$k-1$个工程的剩余的资源总数为：$optx = optx - d_k(optx)$

于是，由此回溯，得到分配给前面各个工程的资源数的递推关系式：

$$\begin{cases} optq_i = d_i(optx) \\ optx = optx - optq_i \qquad i=k,k-1,\ldots,1 \end{cases}$$

算法设计策略——动态规划（Dynamic Programming）

由上面的决策过程知道，求解资源分配问题划分为下面4步：

（1）对各个阶段i，各个不同份额x的资源，计算$f_i(x)$及$d_i(x)$。

（2）计算各阶段的最大利润g_i、获得此最大利润的分配份额q_i。

（3）计算全局的最大利润$optg$、总的资源分配份额optx及最大工程项目数k。

（4）按递推式计算各个工程的最优分配资源数。

算法设计策略——动态规划（Dynamic Programming）

例如：有8个资源，分配给3个工程，其利润函数如下。求资源的最优分配方案。

x	0	1	2	3	4	5	6	7	8
$G_1(X)$	0	4	26	40	45	50	51	52	53
$G_2(X)$	0	5	15	40	60	70	73	74	75
$G_3(X)$	0	5	15	40	80	90	95	98	100

第一步，求各个阶段不同分配份额时的最大利润，以及各个工程在此利润下的分配资源数。

i=1，即第1阶段，只把资源分配给第一个工程，于是有：

x	0	1	2	3	4	5	6	7	8
$f_1(x)$	0	4	26	40	45	50	51	52	**53**
$d_1(x)$	0	1	2	3	4	5	6	7	**8**

算法设计策略——动态规划（Dynamic Programming）

i=2，即第2阶段，把8个资源分配给第一、二个工程，于是有：

x=0时，$f_2(x)=0, d_2(x)=0$

x=1时，$f_2(x)=\max\{G_2(0)+f_1(1), G_2(1)+f_1(0)\}=\max\{4,5\}=5$　　$d_2(x)=1$

x=2时，$f_2(x)=\max\{G_2(0)+f_1(2), G_2(1)+f_1(1), G_2(2)+f_1(0)\}=\max\{26,9,15\}=26$

$d_2(x)=0$

类似可以同样计算出x=3,4,...,8时的$f_2(x)$和$d_2(x)$值。各值如下：

x	0	1	2	3	4	5	6	7	8
$f_2(x)$	0	5	26	40	60	70	86	100	**110**
$d_2(x)$	0	1	0	0	4	5	4	4	**5**

i=3，即第3阶段，把8个资源分配给第一、二、三个工程，于是有：

x=0时，$f_3(x)=0$，$d_3(x)=0$

x=1时，$f_3(x)=\max\{G_3(0)+f_2(1), G_3(1)+f_2(0)\}=\max\{5,5\}=5$　　$d_3(x)=0$

x=2时，$f_3(x)=\max\{G_3(0)+f_2(2), G_3(1)+f_2(1), G_3(2)+f_2(0)\}$

$=\max\{26,10,15\}=26$　　$d_3(x)=0$

类似可以同样计算出x=3,4,...,8时的$f_3(x)$和$d_3(x)$值。各值如下：

x	0	1	2	3	4	5	6	7	8
$f_3(x)$	0	5	26	40	80	90	106	120	**140**
$d_3(x)$	0	0	0	0	4	5	4	4	**4**

第二步，求各个阶段的最大利润，及在此利润下的分配资源数。

g_1=53	g_2=110	g_3=140
q_1=8	q_2=8	q_3=8

第三步，求全局的最大利润optg、最大工程数目k及分配给工程的总的最优分配份额optx。

opgt=max{ 53,110,140 }=140 。 即，全局最大利润为140

optx=8。 即，全局最大利润时分配的资源数是8

k=3 (使利润达到最大的阶段值i)。即，在第3个阶段达到最大利润。

算法设计策略——动态规划（Dynamic Programming）

第四步，求出最大利润时各个工程项目分配的资源数。

第3阶段获得最大利润，利润值为140，分配的资源数为optx=8

此时，分配给第3个工程的资源数为：$optq_3 = d_3(optx) = d_3(8)=4$

于是，分配给前2个工程的资源数 $optx=optx-optq_3=8-4=4$

此时，分配给第2个工程的资源数为：$optq_2=d_2(optx)=d_2(4)=4$

于是，分配给前1个工程的资源数 $optx=optx-optq_2=4-4=0$

此时，分配给第1个工程的资源数为：$optq_1=d_1(optx)=d_1(0)=0$

最后决策结果：分配给第二、三个工程各4个资源，得最大利润140。

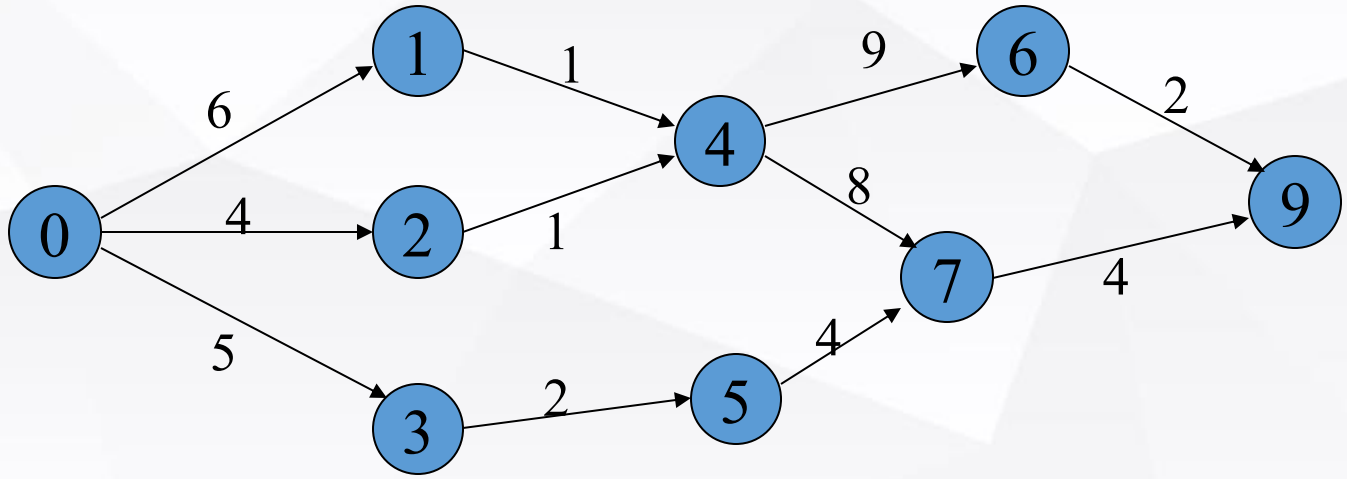
具体算法略！

例4：关键路径问题

*问题描述：AOE网络是一个加权*有向图G=(V,E),顶点表示事件，有向边表示活动，有向边上的权值表示活动的持续时间。利用AOE网络可以进行工程的安排，研究工程的工期及影响工期的关键因素。例如，求完成工程所需的最短时间（工期），即从源点到汇点的最长的路径长度。

研究的前提是AOV 网络；
研究的目的：预测工期、找关键活动。

例如：



算法设计策略——动态规划（Dynamic Programming）

例5：计算矩阵连乘积

问题描述:

在科学计算中经常要计算矩阵的乘积。矩阵A和B可乘的条件是矩阵A的列数等于矩阵B的行数。若A是一个p×q的矩阵，B是一个q×r的矩阵，则其乘积C=A·B是一个p×r的矩阵。其标准计算公式为：

$$C_{ij}=\sum_{k=1}^{q}A_{ik}B_{kj} \qquad \text{其中} 1\le i\le p,\ 1\le j\le r$$

由该公式知计算C=A.B总共需要p·q·r次的数乘，即$O(n^3)$。

现在的问题是，给定n个矩阵$\{A_1,A_2,\ldots,A_n\}$。其中A_i与A_{i+1}是可乘的，i=1,2,…,n-1。要求计算出这n个矩阵的连乘积$A_1A_2\ldots A_n$。

算法设计策略——动态规划（Dynamic Programming）

由于矩阵乘法满足结合律，故连乘积的计算可以有许多不同的计算次序。这种计算次序可以用加括号的方式来确定。

因此，可以先通过一种完全加括号给出一个矩阵连乘积的计算次序，然后再通过反复调用两个矩阵相乘的标准算法计算出矩阵连乘积。

完全加括号的矩阵连乘积可递归地定义为：

(1) 单个矩阵是完全加括号的；即 (A)，简写为： A

(2) 若矩阵连乘积A是完全加括号的，则A可表示为两个完全加括号的矩阵连乘积B和C的乘积并加括号，即A=((B) · (C))。简写为 A=(B · C)

算法设计策略——动态规划（Dynamic Programming）

例如，矩阵连乘积$A_1A_2A_3\ A_4$可以有以下5种不同的完全加括号方式：

$(A_1(A_2(A_3A_4)))$，

$(A_1((A_2A_3)A_4))$，

$((A_1A_2)(A_3A_4))$，

$((A_1(A_2A_3))A_4)$，

$(((A_1A_2)A_3)A_4)$。

每一种完全加括号方式对应于一种矩阵连乘积的计算次序，而这种计算次序与计算矩阵连乘积的数乘计算量有着密切的关系。——即连乘的次序不同，数乘的计算量不同！

算法设计策略——动态规划（Dynamic Programming）

下面看一个例子：计算3个矩阵{A_1，A_2，A_3}的连乘积

A1:10×100　　A2: 100×5　　A3: 5×50

按第一种加括号方式: $((A_1A_2)A_3)$

总共需要10×100×5+10×5×50=7500次的数乘。

按第二种加括号方式: $(A_1(A_2A_3))$

共需要100×5×50+10×100×50=75000次的数乘

第二种加括号方式的数乘计算量是第一种加括号方式的数乘计算量的10倍。由此可见，在计算矩阵连乘积时，加括号方式，即计算次序对数乘计算量有很大影响。

算法设计策略——动态规划（Dynamic Programming）

于是，人们自然会提出矩阵连乘积的最优计算次序问题，即对于给定的相继n个矩阵$\{A_1,A_2,\cdots,A_n\}$，如何确定计算矩阵连乘积$A_1A_2\cdots A_n$的一个计算次序(完全加括号方式)，使得依此次序计算矩阵连乘积需要的数乘次数最少。

设矩阵的维数分别为：$p_0,p_1,p_2,\cdots,p_n$ 其中矩阵A_i的维数为$p_{i-1}\times p_i$ $i=1,2,3,\cdots,n$。p_0,p_1为第1个矩阵的行列值，$p_2,\cdots,p_n$ 为其余各矩阵的列值。——满足相乘的条件！

注：在分治策略中，介绍了Strassen7小矩阵乘法，其时间复杂性为$O(n^{\log 7})=O(n^{2.81})$。目前计算两个n阶方阵相乘最好的计算时间上界是$O(n^{2.367})$。但无论如何，所需的乘法次数总随两个矩阵的阶的增加而递增。在这里，只考虑采用标准公式计算两个矩阵的乘积。

算法设计策略——动态规划（Dynamic Programming）

解这个问题的最容易想到的方法是穷举搜索法。也就是列出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘计算量，然后找出最小者。然而，这样做计算量太大。

事实上，对于n个矩阵的连乘积，设有P(n)个不同的计算次序。由于我们可以首先在第k个和第k+1个矩阵之间将原矩阵序列分为两个矩阵子序列，k=1,2,…,n-1；然后分别对这两个矩阵子序列完全加括号；最后对所得的结果加括号，得到原矩阵序列的一种完全加括号方式。所以关于P(n)，有递推式如下：

$$P(n)=\begin{cases}1 & ,n=1\\ \sum_{k=1}^{n-1}P(k)P(n-k) & ,n\geq 2\end{cases}$$

用数学知识解此递归方程，发现：P(n)实际上是Catalan (卡特郎)数列，即P(n)=C(n-1)，其中，

$$C(n)=\frac{1}{n+1}\binom{2n}{n}=\Omega\left(\frac{4^n}{n^{3/2}}\right)$$

也就是说，P(n)随着矩阵个数n的增长而指数增长。因此，穷举搜索法不是一个有效算法。

于是，可以考虑用动态规划法解矩阵连乘积的最优计算次序问题。此问题是动态规划的典型应用之一。

按以下步骤进行：

算法设计策略——动态规划（Dynamic Programming）

1. 分析最优子结构性质

设计求解具体问题的动态规划算法的第一步是刻画该问题的最优子结构特征。

首先，为方便起见，将矩阵连乘积$A_iA_{i+1}\cdots A_j$简记为$A_{i\cdots j}$。现要求$A_{1\cdots n}$的一个最优次序。设这个计算次序在矩阵A_k和A_{k+1}之间将矩阵链断开，$1\leqslant k<n$，则完全加括号方式为$((A_1\cdots A_k)(A_{k+1}\cdots A_n))$。照此，要先计算$A_{1\cdots k}$和$A_{k+1\cdots n}$，然后，将所得的结果相乘才得到$A_{1\cdots n}$。显然其数乘总计算量为计算$A_{1\cdots k}$的数乘计算量加上计算$A_{k+1\cdots n}$的数乘计算量，再加上$A_{1\cdots k}$与$A_{k+1\cdots n}$相乘的数乘计算量。

这个问题的一个关键特征是：计算$A_{1\cdots n}$的一个最优次序所包含的计算矩阵子链$A_{1\cdots k}$和$A_{k+1\cdots n}$的次序也是最优的。 ——最优子结构性质。

证明：自己完成。

根据该问题的指标函数的特征也可以知道该问题满足最优化原理。另外，该问题显然满足无后向性，因为前面的矩阵链的计算方法和后面的矩阵链的计算方法无关。

2. 建立递归关系

设计非标准的动态规划算法的第二步是递归地定义最优值。

对于矩阵连乘积的最优计算次序问题，假设计算$A_{i\cdots j}$，$1\leqslant i\leqslant j\leqslant n$，所需的最少数乘次数为m[i,j]，原问题的最优值为m[1,n]。于是：

(1) 当i=j时，$A_{i\cdots j}=A_i$为单一矩阵，无需计算，因此：m[i,i]=0，i=1,2,…,n ；

连乘矩阵链：A_i　　i=1,2,…,n;

完全加括号：(A_i)　　i=1,2,…,n;

2. 建立递归关系

(2) 当i<j时，可利用最优子结构性质来计算m[i,j]。

若计算$A_{i\cdots j}$的最优次序在A_k和A_{k+1}之间断开，即若k(i≤k<j) 已知，即

连乘矩阵链：$A_i \quad A_{i+1} \cdots A_k \quad A_{k+1} \quad A_{k+2}\cdots \quad A_j$

各矩阵维数：$p_{i-1}\times p_i \quad p_i\times p_{i+1} \quad p_{k-1}\times p_k \quad p_k\times p_{k+1} \quad p_{k+1}\times p_{k+2} \quad p_{j-1}\times p_j$

完全加括号：$(A_i \quad A_{i+1} \cdots A_k)\ (A_{k+1} \quad A_{k+2}\cdots \quad A_j)$

则：$m[i,j]=m[i,k]+m[k+1,j]+p_{i-1}p_kp_j$

k还未定,需要规划出使这种完全加括号形式的数乘计算量最小的那个。即$k\in\{i,i+1,\cdots,j-1\}$，它是这j-i个位置中计算量达到最小的那一个位置。从而m[i,j]可以递归地定义为：

$$m[i,j]=\begin{cases}0 & ,i=j\\ \min\limits_{i\le k<j}\{m[i,k]+m[k+1,j]+p_{i-1}p_kp_j\} & ,i<j\end{cases}$$

2. 建立递归关系

m[i,j]给出了最优值，即计算$A_{i\cdots j}$所需的最少数乘次数。同时还确定了计算$A_{i\cdots j}$的最优次序中的断开位置k_{min}，也就是说，对于这个k_{min}有:

$$m[i,j] = m[i,k_{min}] + m[k_{min}+1,j] + p_{i-1}p_{kmin}p_j$$

同样，可以用一个二维数组s[n:n]记录，即将对应于m[i,j]的断开位置k_{min}记录在s[i,j]中，则在计算出最优值m[i,j]后，就可以由s[i,j]记录的信息构造出相应的最优解（轨迹）。

3. 计算最优值

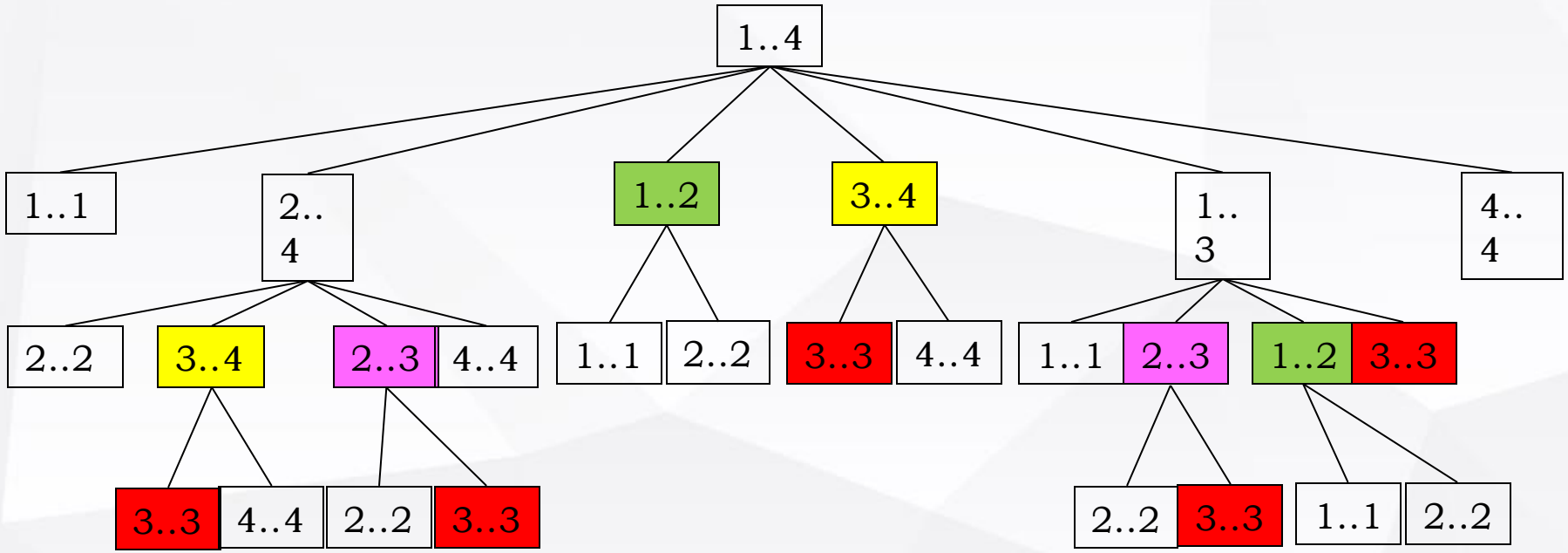
根据m[i,j]的递归定义(2.1)，容易写一个普通的递归程序来计算m[1,n]。 如下：

```
int RecurMatrixChain(int i,int j)
{    if(i=j)  return 0;
          //初值设定，第1种加括号方式从i开始
     u=RecurMatrixChain(i,i)+RecurMatrixChain(i+1,j)
          +p[i-1]*p[i]*p[j]; s[i,j]=i;
        //其他各种加括号方式
     for(int k=i+1;k<j;k++)
        { t=RecurMatrixChain(i,k)+RecurMatrixChain(k+1,j)
        +p[i-1]*p[k]*p[j];
       if (t<u) {  u=t; s[i,j]=k; }
      }
     return u;
}
```

算法设计策略——动态规划（Dynamic Programming）

但是，这个算法的效率如何呢?

先看一个用该算法求解算$A_{1...4}$的过程。该过程可以用递归树表示出来。如下图。



从该图可以看出，许多子问题被重复计算！（标识为同一颜色的子问题）

算法设计策略——动态规划（Dynamic Programming）

事实上，可以证明该算法的计算时间T(n)有指数下界。设算法中判断语句和赋值语句花费常数时间，则由算法的递归部分可得关于T(n)的递归不等式如下：

$$\begin{cases} T(n) \geq 1 & ,n = 1 \\ T(n) \geq 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) & ,n > 1 \end{cases}$$

因此，当n>1时，$T(n) \geq 1 + (n-1) + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k) = n + 2\sum_{i=1}^{n-1} T(i)$

据此，可用数学归纳法证明

$T(n) \geqslant 2^{n-1} = \Omega(2^n)$。

因此，简单地递归求解算法将耗费指数计算时间。

然而，我们注意到，在递归计算过程中，不同的子问题个数只有$\Theta(n^2)$个。事实上，对于 $1 \leqslant i \leqslant j \leqslant n$ 不同的有序对 (i,j) 对应于不同的子问题。因此，不同子问题的个数最多只有：

$$C_n^2 + n = (n^2+n)/2 = \Theta(n^2)$$

由此可见，在递归计算时，许多子问题被重复计算多次。这就是应用动态规划策略的典型特征。

算法设计策略——动态规划（Dynamic Programming）

前面提到，有两种方案：

其一：自底向上递推，即先求子问题的解，稍复杂问题求解时直接使用子问题的解（用到的子问题必须已经有求解结果）。

这就要求必须遵循某种顺序去求解。

其二：自顶向下递归，但使用备忘录，即把已经求解出的子问题的解记录下来，只有是不重复的子问题时才去递归求解。

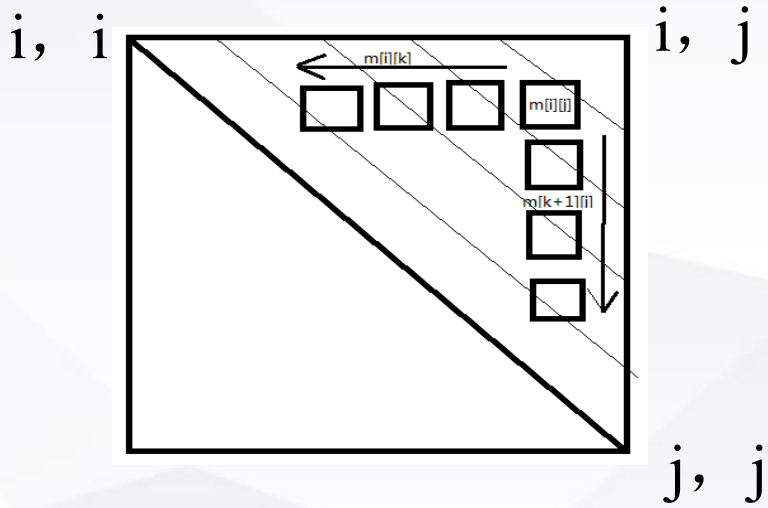
下面先看第一种解决方案： 分析出求解子问题的顺序。

算法设计策略——动态规划（Dynamic Programming）

前面分析知道：

要求计算m[i,j]时，所有m[i,k]和m[k+1,j](i≤k<j)必须都已经计算出来。即要求

m[i,i],m[i,i+1],…m[i,j-1]和m[i+1,j],m[i+2,j],…m[j-1,j]都已计算出来。

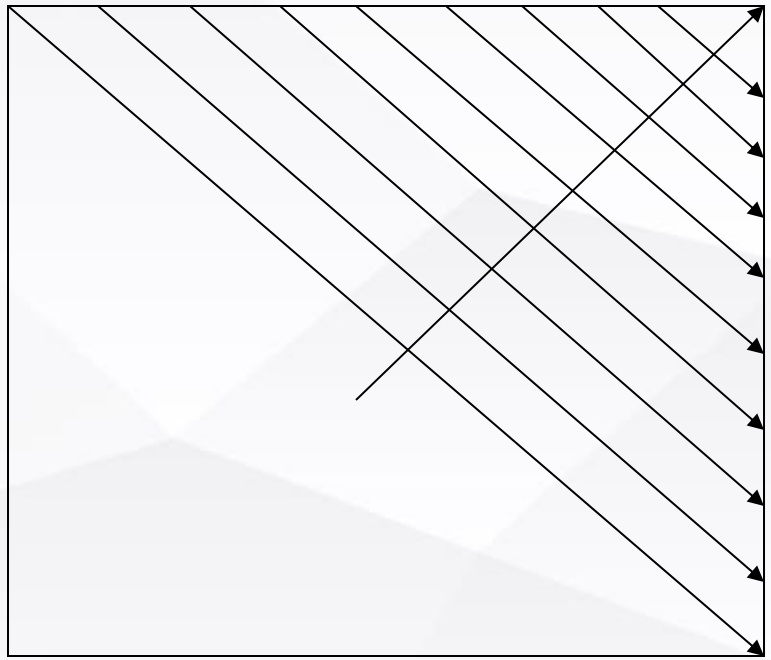


同时把使下式： $m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_kp_j$

达到最优（小）的断开位置记录下来，s[i,j] = k，$1 \leqslant i \leqslant j \leqslant n$ 。

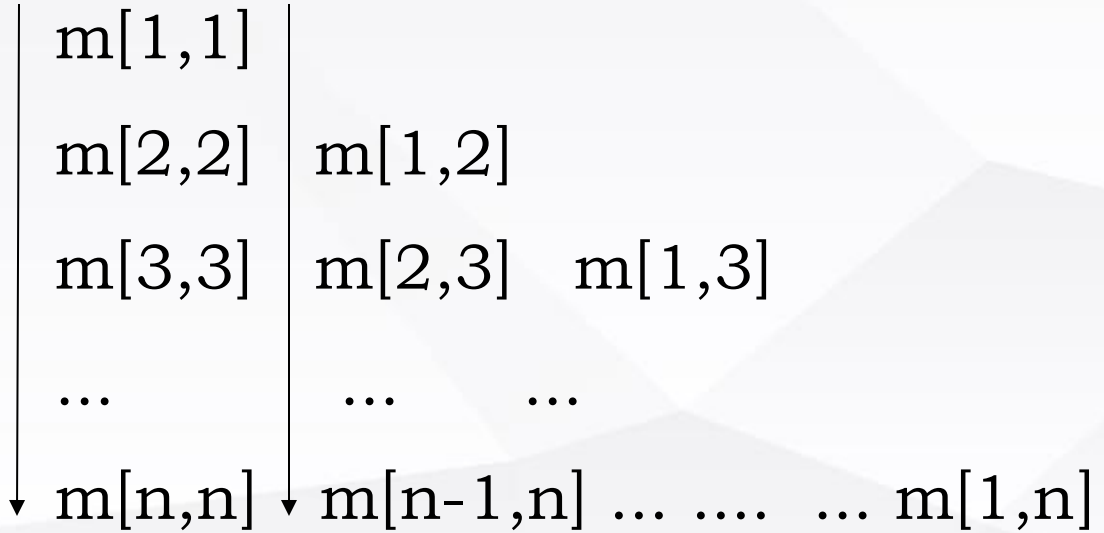
算法设计策略——动态规划（Dynamic Programming）

采用如下的递推计算步骤，可以保证这个计算顺序：首先令m的主对角线元素m[i,i]=0,s[i,i]=i　(i=1,2,…n),然后从主对角线上一条次对角线开始，依次计算m的n-1条对角线元素，并在s[i,j]中记录使得m[i,j]=m[i,k]＋m[k+1,j]+$p_{i-1}p_kp_j$取最小值的k。



算法设计策略——动态规划（Dynamic Programming）

即，首先计算出m[i,i]=0,i=1,2,…n,然后根据递归式，按矩阵链长递增的方式依次计算m[i,i+1],i=1,2,…,n-1(矩阵链长度为2)；m[i,i+2],i=1,2,…,n-2(矩阵链长度为3)；……。在计算m[i,j]时，只用到已计算出的m[i,k]和m[k+1,j]。即按照



的顺序根据公式(2.1)计算m[i,j]。

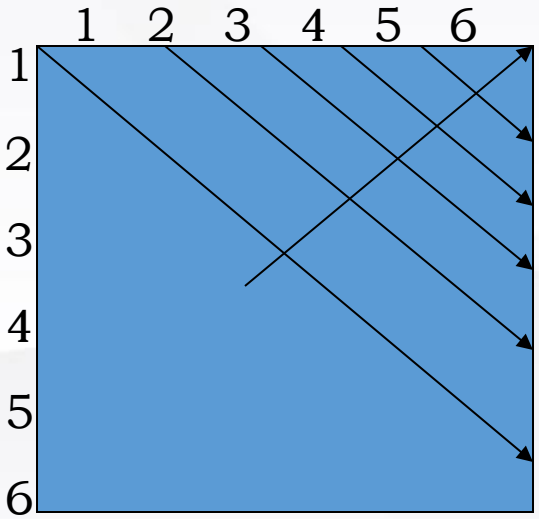
算法设计策略——动态规划（Dynamic Programming）

例如，计算矩阵连乘积$A_1A_2A_3A_4A_5A_6$，其中各矩阵的维数分别为：

A_1：30×35　　A_2：35×15　　A_3：15×5

A_4：5×10　　A_5：10×20　　A_6：20×25

计算m[i,j]先后次序如下图所示；计算结果m[i,j]和s[i,j]如下图：



计算次序

p_0	p_1	p_2	p_3	p_4	p_5	p_6
30	35	15	5	10	20	25

m[1,1]=0, m[2,2]=0, m[3,3]=0,

m[4,4]=0, m[5,5]=0, m[6,6]=0

算法设计策略——动态规划（Dynamic Programming）

$m[1,2]=m[1,1]+m[2,2]+p_0p_1p_2 =0+0+30\times35\times15=15750$ k=1

$m[2,3]=m[2,2]+m[3,3]+p_1p_2p_3 =0+0+35\times15\times5=2625$ k=2

$m[3,4]=m[3,3]+m[4,4]+p_2p_3p_4 =0+0+15\times5\times10=750$ k=3

$m[4,5]=m[4,4]+m[5,5]+p_3p_4p_5 =0+0+5\times10\times20=1000$ k=4

$m[5,6]=m[5,5]+m[6,6]+p_4p_5p_6 =0+0+10\times20\times25=5000$ k=5

$m[1,3]=\min\{ m[1,1]+m[2,3]+p_0p_1p_3 ,m[1,2]+m[3,3]+p_0p_2p_3 \}$

$=\min\{0+2625+5250 ,15750+0+2250\}=\min\{7875,18000\}=7875$

$m[2,4]=\min\{m[2,2]+m[3,4]+p_1p_2p_4 ,m[2,3]+m[4,4]+p_1p_3p_4 \}$

$=\min\{0+750+5250,2625+0+1750\}=\min\{6000,4375\} =4375$

$m[3,5]=\min\{m[3,3]+m[4,5]+p_2p_3p_5 ,m[3,4]+m[5,5]+p_2p_4p_5\}$

$=\min\{0+1000+1500,750+0+3000\}=\min\{2500,3750\} =2500$

$m[4,6]=\min\{ m[4,4]+m[5,6]+p_3p_4p_6 ,m[4,5]+m[6,6]+p_3p_5p_6 \}$

$=\min\{0+5000+1250 ,1000+0+2500\}=\min\{6250,3500\}=3500$

算法设计策略——动态规划（Dynamic Programming）

……

$m[1,6]=\min\{ m[1,1]+m[2,6]+p_0p_1p_6 ,m[1,2]+m[3,6]+p_0p_2p_6$
$m[1,3]+m[4,6]+p_0p_3p_6, m[1,4]+m[5,6]+p_0p_4p_6$
$m[1,5]+m[6,6]+p_0p_5p_6 \}$
$=\min\{0+10500+26250 ,15750+5735+11250$
$7875+3500+3750,9375+3500+7500$
$11875+0+15000 \}$
$=\min\{36750,32735,15125,20375,26875\}=15125$ k=3

最终结果如右：

m[i][j]

	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

算法设计策略——动态规划（Dynamic Programming）

同时记录得到最小乘次数的s[i,j]，如下：

	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

s[i][j]

算法如下：

算法设计策略——动态规划（Dynamic Programming）

```
public static void MatrixChain(int [ ]p, int [ ][ ]m, int[ ][ ]s)
{
    int n=p.length-1;
    for(int i=1;i<=n;i++) m[i][i]=0;
    for(int r=2;r<=n;r++)
      for(int i=1;i<=n-r+1;i++)
      { int j=i+r-1;
        m[i][j]=m[i+1][j]+p[i-1]*p[i]*p[j];
        s[i][j]=i;
        for(int k=i+1;k<j;k++)
        {  int t=m[i][k]+m[k+1][j]+p[i-1]*p[k]*p[j];
            if(t<m[i][j])
              {  m[i][j]=t;
                 s[i][j]=k;   }
        }
}
```

该算法的计算时间上界为$O(n^3)$。由此可见，动态规划算法比穷举搜索法要有效得多。

4. 构造最优解

动态规划算法的第四步是构造问题的最优解。上面的算法只给出了最优值，并未给出最优解，即通过该算法，只是知道最少数乘次数，还不知道具体应按什么次序做矩阵乘法才能达到最少的数乘次数。

事实上。算法已记录了构造一个最优解所需要的全部信息。那么如何构造最优解呢?

算法设计策略——动态规划（Dynamic Programming）

s[i][j]中的数k告诉我们计算矩阵链$A_{i\cdots j}$的最佳方式应在矩阵A_k和A_{k+1}之间断开，即最优的加括号方式应为$(A_{i\cdots k})(A_{k+1\cdots j})$。因此，从s[1,n]记录的信息可知计算$A_{1\cdots n}$的最优加括号方式为 $(A_{1\cdots s[1,n]})(A_{s[1,n]+1\cdots n})$。而计算$A_{1\cdots s[1,n]}$的最优加括号方式为$(A_{1\cdots s[1,s[1,n]]})(A_{s[1,s[1,n]]+1\cdots s[1,n]})$。同理可以确定计算$A_{s[1,n]+1\cdots n}$的最优加括号方式在s[s[1,n]+1,n]处断开。…照此递推下去，最终可以确定$A_{s[1,n]+1\cdots n}$的最优完全加括号方式，即构造出问题的一个最优解。

下面的算法Traceback按上面算法计算出的断点矩阵s指示的加括号方式输出计算$A_{i\cdots j}$的最优计算次序。

算法设计策略——动态规划（Dynamic Programming）

```
public static void traceback(int [][]s,int i, int j)
{
    if(i==j) return;
      traceback(s,i,s[i][j]);
      traceback(s,s[i][j]+1,j);
      system.out.println("Multiply A"+i+","+s[i][j]+
                         "and A"+(s[i][j]+1)+","+j);
}
```

```
矩阵的最少计算次数为：15125
矩阵最优计算次序为：
Multiply A2,2 and A3,3
Multiply A1,1 and A2,3
Multiply A4,4 and A5,5
Multiply A4,5 and A6,6
Multiply A1,3 and A4,6
请按任意键继续. . .
```

要计算$A_{1\ldots n}$只要调用traceback(1,n,s)即可。对于上面的例子，调用为traceback(1,6,s),可输出最优计算次序$((A_1(A_2A_3))((A_4A_5)A_6))$,即

S[1,6]=3,即k=3 $A_{1..3}$ $A_{4..6}$

S[1,3]=1,即k=1 $A_{1..1}$ $A_{2..3}$

S[4,6]=5,即k=5 $A_{4..5}$ $A_{6..6}$

S[2,3]=2,k=2 $A_{2..2}$ $A_{3..3}$

S[4,5]=5,k=5 $A_{4..4}$ $A_{5..5}$

算法设计策略——动态规划（Dynamic Programming）

计算最优值的第二种解决方案： 备忘录

备忘录方法为每个已经计算的子问题建立备忘录即保存子问题的计算结果以备需要时引用，从而避免了相同子问题的重复求解。

备忘录方法的控制结构与直接递归方法的控制结构相同，区别仅在于备忘录方法为每个解过的子问题建立了备忘录以备需要时查看，避免了相同子问题的重复求解。

下面的算法是解矩阵连乘积最优计算次序问题的备忘录方法。

```
int MemoizedMatrixChain(int n,int **m,int **s)
{ for (int i=1;i<=n;i++)
    for(int j=i;j<=n;j++)
       m[i,j]=0;
   return LookupChain(1,n);
}
```

算法设计策略——动态规划（Dynamic Programming）

```
int LookupChain(int i,int j)
{
   if(m[i][j]>0) return m[i][j];  //计算出来后就不再递归计算了
   if(i==j) return 0;
   int u=LookupChain(i,i)+LookupChain(i+1,j)+p[i-1]*p[i]*p[j];
   s[i][j]=i;
   for(int k=i+1;k<j;k++)
   { int t=LookupChain(i,k)+LookupChain(k+1,j)+p[i-1]*p[k]*p[j];
      if(t<u)  {  u=t; s[i][j]=k;}       }
    m[i][j]=u;
    return u;
 }
```

算法设计策略——动态规划（Dynamic Programming）

与动态规划算法一样，备忘录算法耗时$O(n^3)$。事实上，共有$O(n^2)$个备忘记录项m[i,j]，i=1,2,…,n，j=i,i+1,…n，这些记录项的初始化耗费$O(n^2)$时间。每个记录项只填入一次，每次填入时，不包括填入其他记录项的时间，共耗费O(n)。因此，LookupChain填入$O(n^2)$个记录项总共耗费$O(n^3)$计算时间。由此可见，通过使用备忘录技术，直接递归算法的计算时间从仍$O(2^n)$降至$O(n^3)$。

从算法可以看出，该算法的有效性依赖于问题本身所具有的三个重要性质：

- 最优子结构性质
- 无后向性
- 子问题重叠性质

例6.最长公共子序列问题LCS(Longest Common Subsequeue)

问题描述:

一个给定序列的子序列(Subsequeue)是在该序列中删去若干元素后得到的序列。确切地说，若给定序列$X=<x_1, x_2,\cdots, x_m>$，则另一序列$Z=<z_1, z_2,\cdots, z_k>$是X的子序列是指存在一个严格递增的下标序列 $<i_1, i_2,\cdots, i_k>$，使得对于所有$j=1,2,\cdots,k$有

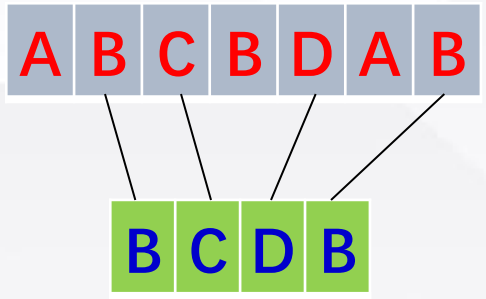
即按原主序列相对次序构成!

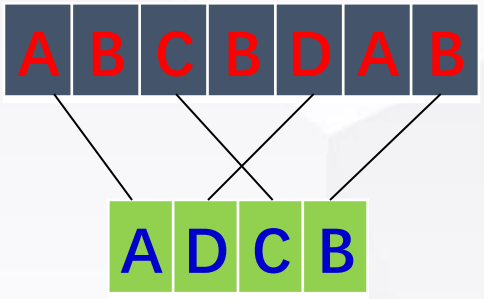
例如，序列Z=<B,C,D,B>是序列X=<A,B,C,B,D,A,B>的子序列，相应的递增下标序列为<2,3,5,7>。

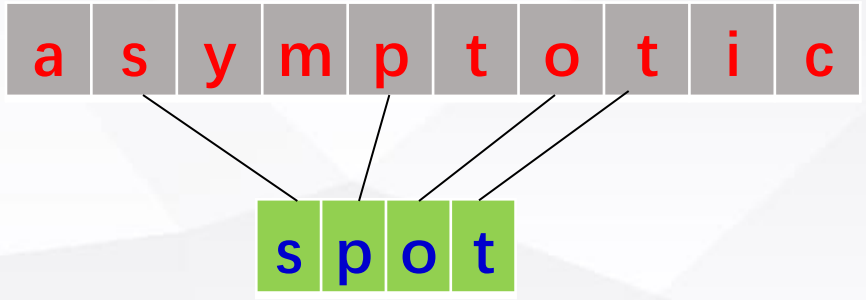
$i_1=2, i_2=3, i_3=5, i_4=7$

j=1,2,3,4

算法设计策略——动态规划（Dynamic Programming）



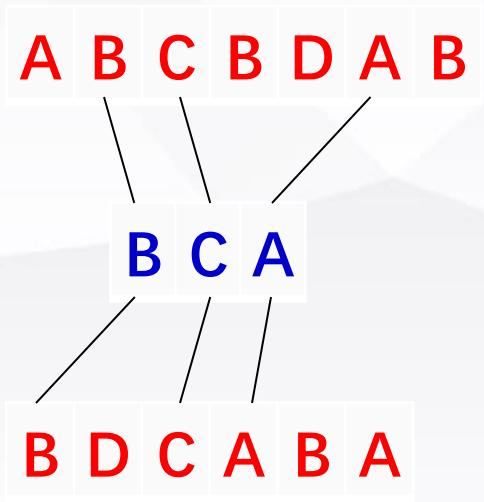


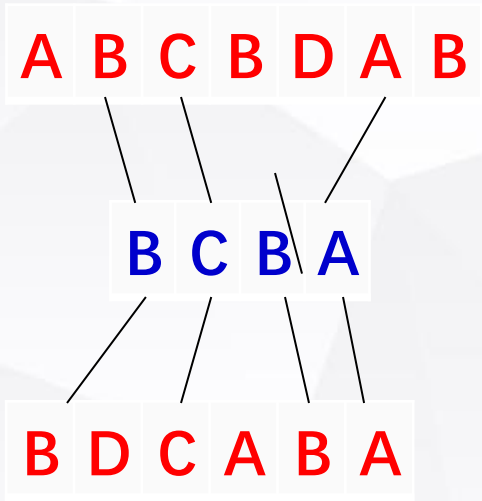


算法设计策略——动态规划（Dynamic Programming）

给定两个序列*X*和*Y*，当另一序列Z既是X的子序列又是Y的子序列时，称Z是序列X和Y的公共子序列(Common Subsequeue)。

例如，若X=<A, B, C, B, D, A, B>和Y=<B, D, C, A, B, A>，则序列<B, C, A>是X和Y的一个公共子序列，序列<B, C, B, A>也是X和Y的一个公共子序列。而且，后者是X和Y的一个最长公共子序列，因为X和Y没有长度大于4的公共子序列。

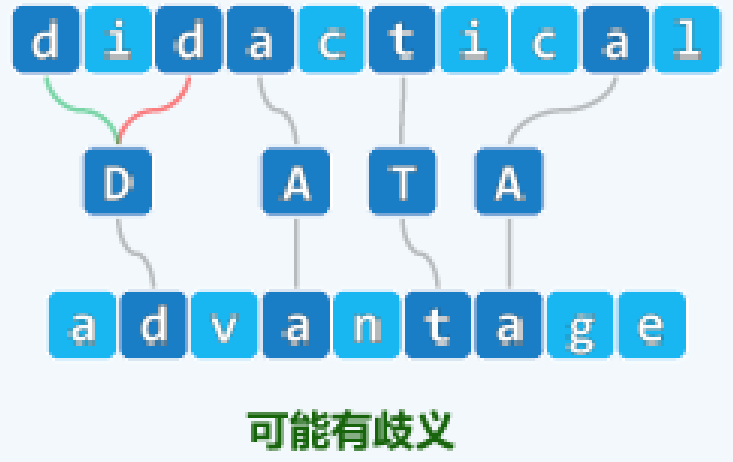




算法设计策略——动态规划（Dynamic Programming）

最长公共子序列(LCS)问题：给定两个序列X=<x_1, x_2, …, x_m>和Y=<y_1, y_2, … , y_n>，要求找出X和Y的一个最长公共子序列。





DATA
DANA

算法设计策略——动态规划（Dynamic Programming）

解最长公共子序列问题时最容易想到的算法是穷举搜索法，即对X的每一个子序列，检查它是否也是Y的子序列，从而确定它是否为X和Y的公共子序列，并且在检查过程中选出最长的公共子序列。

X的所有子序列都检查过后即可求出X和Y的最长公共子序列。可以看出，对于一个长度为m的序列X，其每一个子序列都对应于下标集合{1, 2, …, m}的一个子集，因此，X共有2^m个不同子序列，从而穷举搜索法需要指数时间。

因此我们考虑用动态规划来设计解决的算法。下面的分析可以看到，使用动态规划策略可有效地解此问题。

算法设计策略——动态规划（Dynamic Programming）

1. 最优子结构特性

最长公共子序列问题具有最优子结构性质，有如下定理：

定理：

设$X=<x_1, x_2, \cdots, x_m>$和$Y=<y_1, y_2, \cdots, y_n>$是两个序列，$Z=<z_1, z_2, \cdots, z_k>$是它们的一个最长公共子序列，则

(1) 若$x_m=y_n$，则$z_k=x_m=y_n$且Z_{k-1}是X_{m-1}和Y_{n-1}的最长公共子序列；

(2) 若$x_m \neq y_n$且$z_k \neq x_m$，则Z是X_{m-1}和Y的最长公共子序列；

(3) 若$x_m \neq y_n$且$z_k \neq y_n$，则Z是X和Y_{n-1}的最长公共子序列。

其中：$X_{m-1}=<x_1, x_2, \cdots, x_{m-1}>$

$Y_{n-1}=<y_1, y_2, \cdots, y_{n-1}>$

$Z_{k-1}=<z_1, z_2, \cdots, z_{k-1}>$

证明：反证法

（1） 若$x_m = y_n$,但$z_k \neq x_m$

把x_m加入到Z中，则$<z_1, z_2, \cdots, z_k, x_m>$是X和Y的长度为k+1的公共子序列。这与Z是X和Y的一个最长公共子序列矛盾。因此，必有$z_k = x_m = y_n$。

由此可知：Z_{k-1}是X_{m-1}和Y_{n-1}的一个长度为k-1的公共子序列。

因为：若X_{m-1}和Y_{n-1}有一个长度大于k-1的公共子序列W，则将x_m加在其尾部将产生X和Y的一个长度大于k的公共子序列。此为矛盾。故Z_{k-1}是X_{m-1}和Y_{n-1}的一个最长公共子序列。

（2）由于$z_k \neq x_m$，Z是X_{m-1}和Y的一个公共子序列。

因为：若X_{m-1}和Y有一个长度大于k的公共子序列W，则W也是X和Y的一个长度大于k的公共子序列。这与Z是X和Y的一个最长公共子序列矛盾。由此即知Z是X_{m-1}和Y的一个最长公共子序列。

（3）与 （2）类似。

这个定理告诉我们，两个序列的最长公共子序列包含了这两个序列的前缀的最长公共子序列。因此，最长公共子序列问题具有最优子结构性质。

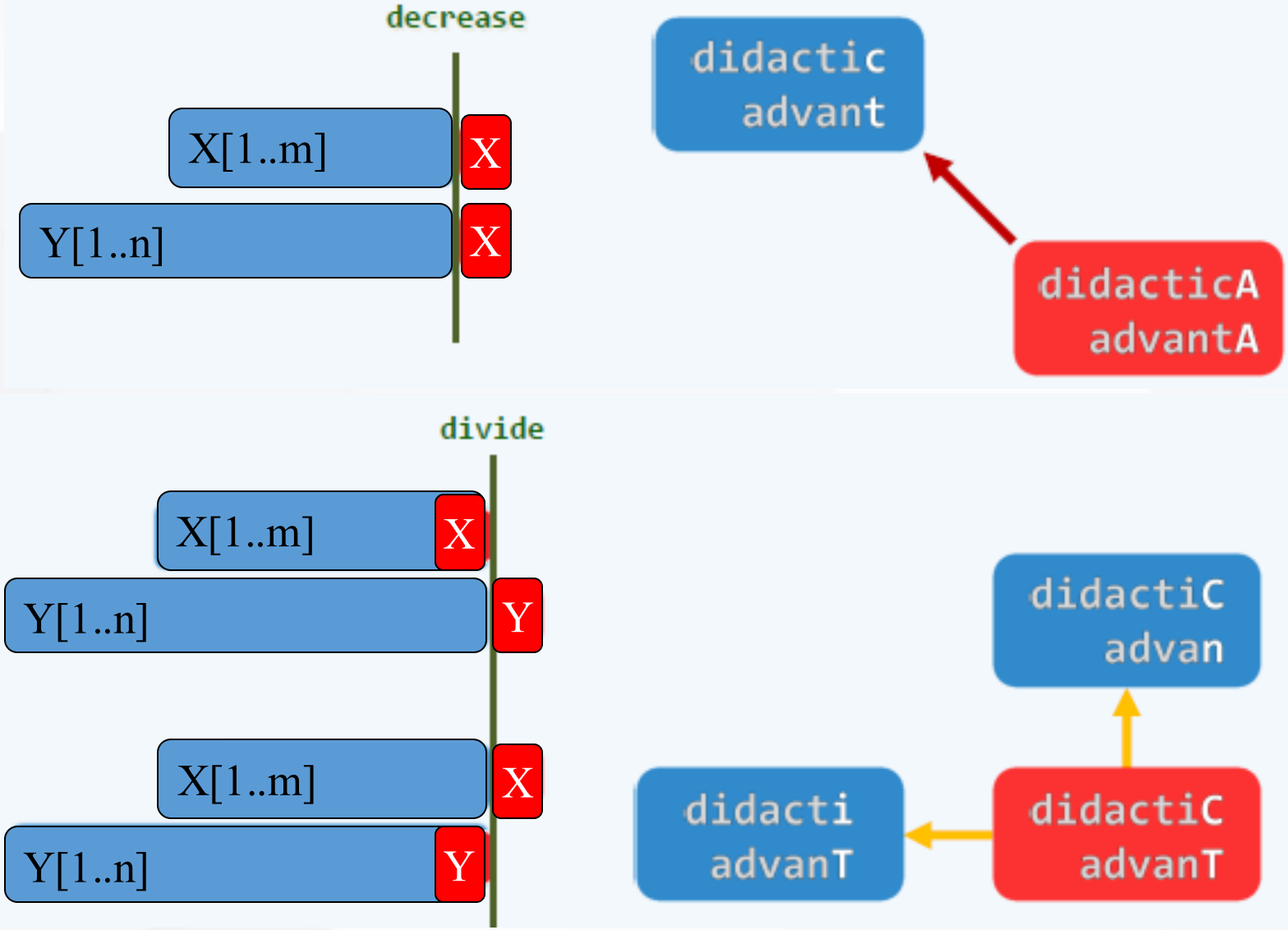
2.子问题的递归结构

由最长公共子序列问题的最优子结构性质可知，要找出$X=<x_1, x_2, \cdots, x_m>$和$Y=<y_1, y_2, \cdots, y_n>$的最长公共子序列，可按以下方式递归地进行：

当$x_m=y_n$时，找出X_{m-1}和Y_{n-1}的最长公共子序列，然后在其尾部加上$x_m(=y_n)$即可得X和Y的一个最长公共子序列。

当$x_m \neq y_n$时，必须解两个子问题，即找出X_{m-1}和Y的一个最长公共子序列及X和Y_{n-1}的一个最长公共子序列。这两个公共子序列中较长者即为X和Y的一个最长公共子序列。

算法设计策略——动态规划（Dynamic Programming）



要注意：由此递归结构容易看到最长公共子序列问题具有子问题重叠性质。例如，在计算X和Y的最长公共子序列时，可能要计算出X和Y_{n-1}、X_{m-1}和Y的最长公共子序列。而这两个子问题都包含一个公共子问题，即计算X_{m-1}和Y_{n-1}的最长公共子序列。

算法设计策略——动态规划（Dynamic Programming）

于是，可以建立子问题的最优值的递归关系。

用c[i,j]记录序列X_i和Y_j的最长公共子序列的长度。

其中$X_i=<x_1, x_2, \cdots, x_i>$，$Y_j=<y_1, y_2, \cdots, y_j>$。

当i=0或j=0时，空序列是X_i和Y_j的最长公共子序列，故c[i,j]=0。

其他情况下，由定理可建立递归关系如下：

$$c[i,j]=\begin{cases}0 & \text{当}i=0\text{或}j=0\text{时}\\ c[i-1,j-1]+1 & \text{当}i,j>0\text{且}x_i=y_j\text{时}\\ \max(c[i,j-1],c[i-1,j]) & \text{当}i,j>0\text{且}x_i\neq y_j\text{时}\end{cases}$$

3. 计算最优值

直接利用递归式自顶向下容易写出一个计算c[i,j]的递归算法，但其计算时间是随输入长度指数增长的。可以增加备忘录!

也可以采用自底向上的动态规划算法计算最优值，提高算法的效率。

假设输入序列为：$X=<x_1, x_2, \cdots, x_m>$ $Y=<y_1, y_2, \cdots, y_n>$

输出：数组c[0..m ,0..n]和b[0..m ,0..n]。

其中：c[i,j]存放X_i与Y_j的最长公共子序列的长度

b[i,j]记录指示c[i,j]的值是由哪一个子问题的解达到的（决策），针对三种情况，取值分别为 1、 2、 3，这在构造最长公共子序列（得到解轨迹）时要用到。

最后，X和Y的最长公共子序列的长度记录于c[m,n]中。

算法设计策略——动态规划（Dynamic Programming）

```
void LCSLength(int m,int n,char *x,char *y,int **c,Type **b)
{ for(i=1;i<=m;i++) c[i][0]=0;   //j=0
  for(i=1;i<=n;i++) c[0][i]=0;    //i=0
  for(i=1;i<=m;i++)
    for(j=1;j<=n;j++)
    { if(x[i]==y[j])
        { c[i][j]=c[i-1][j-1]+1;    //由c[i-1][j-1]计算c[i][j]
          b[i][j]="1";  }
      else if(c[i-1][j]>c[i][j-1]
              { c[i][j]=c[i-1][j];   //由c[i][j-1]计算c[i][j]
                b[i][j]="2"; }
            else
              {  c[i][j]=c[i][j-1];  //由c[i-1][j]计算c[i][j]
                 b[i][j]="3"; }
    }
  return c[m][n];
}
```

由于每个数组单元的计算耗费$O(1)$时间，算法LCSLength耗时$O(m \times n)$。

4.构造最长公共子序列——最优解轨迹

由算法计算得到的数组c和b，数组元素c[m,n]中记录了问题的最优解值。数组b记录了关于解轨迹的信息，可用于快速构造序列X=<x_1, x_2, …, x_m>和Y=<y_1, y_2, …, y_n>的最长公共子序列。方法如下：

沿着数组b中记录的信息进行解的搜索：开始i=m，j=n;

(1) 当b[i,j]=“1”时，表示X_i与Y_j的最长公共子序列是由X_{i-1}与Y_{j-1}的最长公共子序列在尾部加上x_i得到的子序列；于是输出x_i。i减1，j减1。

(2) 当b[i,j]=“2”时，表示X_i与Y_j的最长公共子序列和X_{i-1}与Y_j的最长公共子序列相同，于是 i减1。

(3) 当b[i,j]=“3”时，表示X_i与Y_j的最长公共子序列和X_i与Y_{j-1}的最长公共子序列相同，于是 j减1。

重复(1)-(3)直到i=0或者j=0为止。 得到的就是解轨迹!

算法设计策略——动态规划（Dynamic Programming）

下面的算法实现根据b的内容打印出X_i与Y_j的最长公共子序列。

```
void LCS(int i,int j,char *x,Type **b)
{    if(i==0 | | j==0)  return;
      if(b[i][j]== “ 1 ”
      {   LCS(i-1,j-1,x,b);
           cout<<x[i];  }
     else if(b[i][j]== “ 2 " LCS(i-1,j,x,b)
           else LCS(i.j-1,x,b);
  end;
```

在算法LCS中，每一次的递归调用使i或j减1，因此算法的计算时间为$O(m+n)$。

算法设计策略——动态规划（Dynamic Programming）

例如：X={A,B,C,B,D,A,B} Y={B,D,C,A,B,A},由算法得出的计算结果如下：

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1
2	0	1	1	1	1	2	2
3	0	1	1	2	2	2	2
4	0	1	1	2	2	3	3
5	0	1	2	2	2	3	3
6	0	1	2	2	3	3	4
7	0	1	2	2	3	4	4

类似计算出所有c[i,j]值

i=0或者j=0有c[0,j]=0或者c[i,0]=0

当i=1,j=1时，x[i]='A',y[j]= 'B' ，即x[i] ≠y[j],于是
c[i-1,j]=c[0,j]=0 c[i,j-1]=c[i,0]=0
因为c[i-1,j]>=c[i,j-1] 则：
c[i,j]=c[i-1,j]=c[0,j]=0,即c[1,1]=0
s[1,1]='2'

同样i=1,j=2时，x[1]= 'A',y[2]= 'D'
即x[i] ≠y[j],于是
c[0,2]=0, c[1,1]=0
因为c[i-1,j]>=c[i,j-1] 则：
c[1,2]=c[0,2]=0,即c[1,2]=0,s[1,2]='2'

算法设计策略——动态规划（Dynamic Programming）

同时得到的b[i,j]值如右：

追溯路径为：

2:i-1　1:i-1,j-1

2:i-1　1:i-1,j-1

3:j-1　1:i-1,j-1

3:j-1　1:i-1,j-1

0

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	2	2	2	1	3	1
2	0	1	3	3	2	1	3
3	0	2	2	1	3	2	2
4	0	1	2	2	2	1	3
5	0	2	1	2	2	2	2
6	0	2	2	2	1	2	1
7	0	1	2	2	2	1	2

Y

X

因此：最长公共子序列长度为c[7,6]=4;最长的公共子序列由其中所有为“1”的项对应的b[i,j]的xi组成，$x_2x_3x_4x_6$即:<B,C,B,A>

例7.0/1背包问题

问题描述

已知一个载重为M的背包和n件物品，物品编号为0-n-1。第i件物品的重量为w_i，如果将第i种物品装入背包将获益p_i，这里$w_i>0, p_i>0$ $(0 \le i<n)$。0/1背包问题是：在物品不能分割、只能整件装入或者不装入的情况下，求一种最佳装载方案使得收益最大。

约束条件：$\sum w_i.x_i \le M$ $x_i \in \{0,1\}$ $(0 \le i<n)$

目标函数：$\sum p_i.x_i$ 最大 $x_i \in \{0,1\}$ $(0 \le i<n)$

通常使用KNAP(0,n-1,M)表示一个问题的实例。

算法设计策略——动态规划（Dynamic Programming）

1. 最优子结构性质

0/1背包的最优解具有最优子结构性质。

设$(x_0,x_1,\ldots x_{n-1})$，$x_i \in \{0,1\}$是0/1背包的最优解，那么$(x_1,x_2,\ldots x_{n-1})$必然是0/1背包子问题的最优解：背包载重M-w_0x_0，共有n-1件物品，第i件物品的重量为w_i，效益为p_i，$w_i>0$, $p_i>0$,$1 \leqslant i < n$。

证明：自己完成

2.子问题的递归结构—动态规划方程

假设给定一个0/1背包问题实例KNAP(0,n-1,M)。$x_i \in \{0,1\}$,$x_i=1$表示将物品装入背包，$x_i=0$表示物品不装入背包。

假设对这些x_i做出决策的次序是$X=\{x_{n-1},x_{n-2},\cdots x_0\}$,对$x_{n-1}$做出决策后，存在两种情况：

（1）$x_{n-1}=1$,即将编号为n-1的物品加入背包，接着求解子问题KNAP(0,n-2,M-w_{n-1})。

（2） $x_{n-1}=0$,即编号为n-1的物品不加入背包，接着求解子问题KNAP(0,n-2,M)。

假设用f(j,X)记 背包载重为X、可供选择的物品有j+1个（依次为第0,1,…,j个）时的最优解值。

对于任意的j，0≤j<n有： $f(j,X) = \max\{ f(j-1,X), f(j-1,X-w_j)+p_j \}$ //第j个装和不装

那么，f(n-1,X)可表示为：$f(n-1,X)=\max\{ f(n-2,X), f(n-2,X-w_{n-1})+p_{n-1} \}$ //第n-1个装和不装

于是得到最优解的递归式如下：

$$f(-1,X)=\begin{cases} -\infty & X<0 \\ 0 & X\geqslant 0 \end{cases}$$

$$f(j,X) = \max\{ f(j-1,X), f(j-1,X-w_j)+p_j \} \qquad 0\leqslant j<n$$

背包容量为M、n个物品的0/1背包问题的最优解值为：。

$$f(n-1,M)= \max\{ f(n-2,M), f(n-2,M-w_{n-1})+p_{n-1} \}$$

根据上面的递归式，很容易写出一个普通的递归算法进行求解！

算法设计策略——动态规划（Dynamic Programming）

例子： 设有0/1背包问题 n=3，$(w_0,w_1,w_2)=(2,3,4)$，$(p_0,p_1,p_2)=(1,2,4)$M=6，求最优解。

f(2,6)=max{ f(1,6), f(1,6-4)+4 } = max{ f(1,6), f(1,2)+4 }=5

f(1,6)=max{ f(0,6), f(0,6-3)+2 } = max{ f(0,6), f(0,3)+2 }=3

f(1,2)=max{ f(0,2), f(0,2-3)+2 } = max{ f(0,2), f(0,-1)+2 } =1

f(0,6)=max{ f(-1,6), f(-1,6-2)+1 } = max{ 0, f(-1,4)+1 } = 1

f(0,3)=max{ f(-1,3), f(-1,3-2)+1 } = max{ 0, f(-1,1)+1 } = 1

f(0,2)=max{ f(-1,2), f(-1,2-2)+1 } = max{ 0, f(-1,0)+1 } = 1

f(0,-1)=max{ f(-1,-1), f(-1,-1-2)+1 } = max{-∞ , -∞ +1 } = -∞

f(-1,x)=0 x≥0 ; f(-1,x)= -∞ x<0;

于是，知道最大收益为 5。 那么具体装载方案是什么样子?

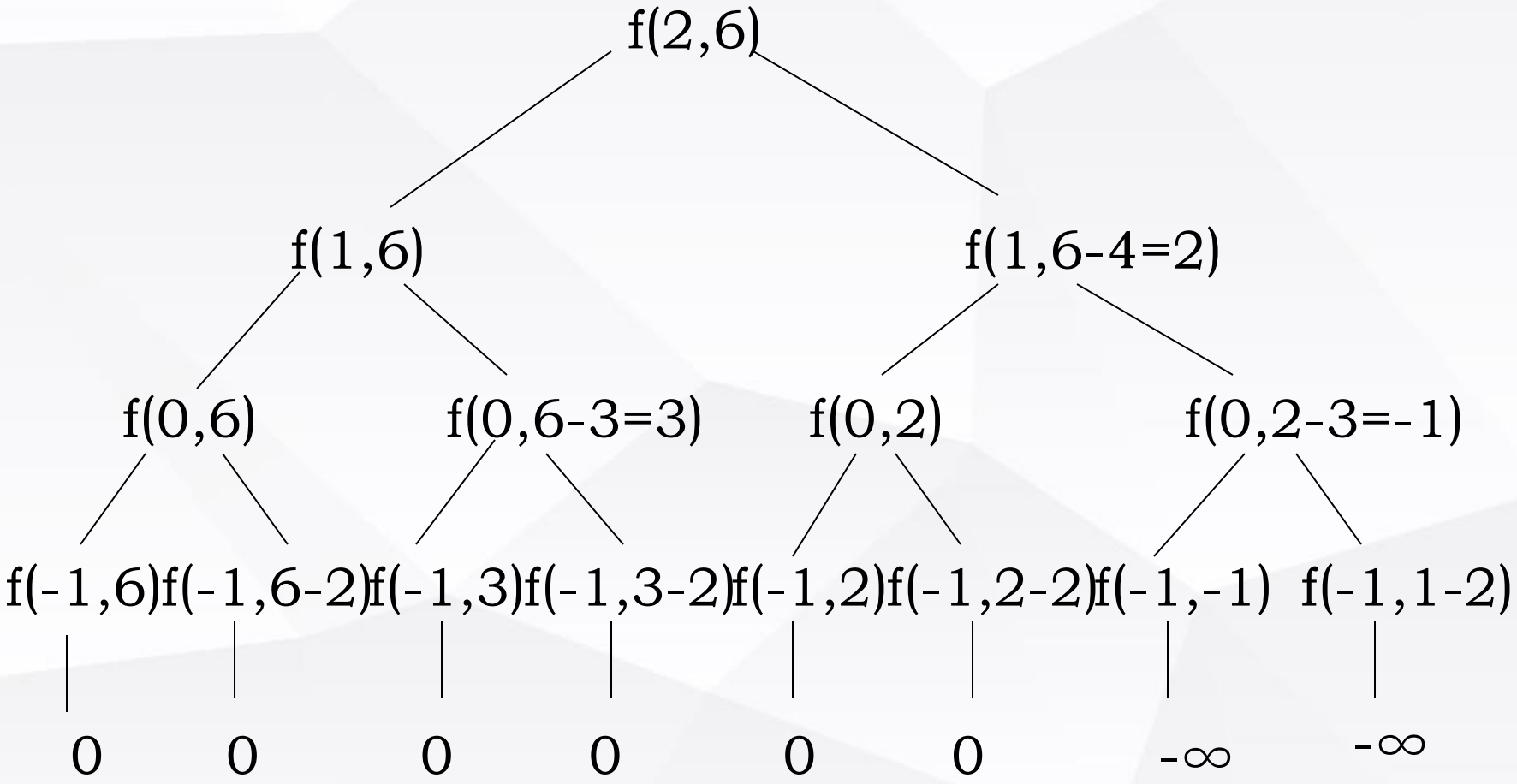
得到解轨迹：

- 从求解最优值过程可以知道，最大效益是在 第j=2个装入时达到的。于是$x_2=1$
- 变成了子问题为f(1,2)，其最大效益为1，这是在第j=1个不装入时达到的。于是$x_1=0$
- 子问题又成为f(0,2)，其最大效益为1，这是在第j=0个装入时达到的。于是$x_0=1$

从而得到装载方案：$(x_0,x_1,x_2) = (1,0,1)$

算法设计策略——动态规划（Dynamic Programming）

普通递归求解的递归树如下：



求最优值的普通的递归算法
请同学们自己写出！

按照普通的递归求解，可以知道该算法的时间复杂性为：

$T(0)=T(1)=c1$

$T(n)=2T(n-1)+c2 \quad n>1$

于是有：$T(n)=O(2^n)$

3. 动态规划算法

从递归过程我们可以看出，解最优值f(2,6)的计算被转化为求解f(1,6)和f(1,2)两个子问题，而这两个子问题又转化为f(0,6) 、f(0,3)、f(0,2)、 f(0,-1)四个子问题，而这四个子问题又转化为8个子问题，这些子问题可以求出解了。然后由子问题的解，按照前面的递归式，求出原问题的最优解。

假设X是满足0≤X≤M条件的整数，我们就可以采用自底向上的递推进行计算。已经求出的子问题的最优值可以用二维数组f[j,k]来保存，0≤j<n，0≤k ≤ M。使其达到最优的方案用b[j]来保存，b[j]怎么取值?

此时必须对X的不同的值计算f[j,k]，因此总的计算量为O(M×n)。具体算法自己写出。

如果物品重量和背包载重为实数，那么子问题的最优解f(j,X)是X 0≤X≤ M的连续函数，上述的方法不可行。

动态规划策略小结

1．动态规划策略一般用于求解“多阶段决策”的最优化问题。

标准和非标准

2. 适用的条件：

（1）最优子结构性质

（2）无后向性：对于确定的x_k，无论p_{1k-1}如何，最优子策略p_{kn}*是唯一确定的，这种性质称为无后向性。阶段划分”是关键，是动态规划的技巧.

（3）重叠子问题

算法设计策略——动态规划（Dynamic Programming）

3. 动态规划策略与分治策略比较：

（1）相同点：

大问题分解为子问题，通过子问题的求解得到原问题的解。

（2）不同点：

分治主要有分解步和组合步，把解组合起来；而动态规划是刻画出子问题最优解和原问题最优解的关系。一般分解为前部或后部子问题。

分治策略不考虑解决冗余，动态规划解决冗余。

算法设计策略——动态规划（Dynamic Programming）

4. 最优化问题的动态规划算法设计技巧性很强，最关键的步骤是：

发现问题和其子问题的最优解的内在关系， 即：列出动态规划方程

动态规划方程一般是递归式

动态规划算法就是求解动态规划方程：

自顶向下（增加备忘录）

自底向上（子问题的求解顺序）

算法设计策略——动态规划（Dynamic Programming）

例题：编辑距离

- 设*a*和*b*是两个字符串。现在要用最少的字符操作次数（最优编辑距离），将字符串*a*编辑为字符串*b*。这里的字符编辑操作共有3种：删除一个字符，插入一个字符或者将一个字符替换另一个字符。
- 例如，*a*="sfdqxbw"，*b*="gfdgw"，结果为4。

算法设计策略——动态规划（Dynamic Programming）

$a \Rightarrow b$

解

- 设计二维动态规划数组dp，其中dp[i][j]表示将a[0..i-1]（1≤i≤m）编辑为b[0..j-1]（1≤j≤n）的最优编辑距离（即最少编辑操作次数）。
- 当b为空串时，要删除a中全部字符得到为b，即dp[i][0]=i（删除a中i个字符，共i次操作）。
- 当a为空串时，要在a中插入b的全部字符得到b，即dp[0][j]=j（向a中插入b的j个字符，共j次操作）。

当两个字符串a、b均不空时，若$a[i-1]=b[j-1]$时，这两个字符不需要任何操作，即$dp[i][j]=dp[i-1][j-1]$。

当$a[i-1]\neq b[j-1]$时，以下3种操作都可以达到目的：

① 将$a[i-1]$替换为$b[j-1]$，有$dp[i][j]=dp[i-1][j-1]+1$（一次替换操作的次数计为1）。

② 在$a[i-1]$字符后面插入$b[j-1]$字符，有$dp[i][j]=dp[i][j-1]+1$（一次插入操作的次数计为1）。

③ 删除$a[i-1]$字符，有$dp[i][j]=dp[i-1][j]+1$（一次删除操作的次数计为1）。

此时$dp[i][j]$取上述三种操作的最小值。

算法设计策略——动态规划（Dynamic Programming）

```
int editdist()                          //求解算法
{   int m=a.size();
    int n=b.size();
    int dp[MAX][MAX];                   //二维动态规划数组
    for (int i=1;i<=m;i++)
        dp[i][0]=i;                     //把a的i个字符全部删除转换为b
    for (int j=1; j<=n;j++)
        dp[0][j]=j;                     //在a中插入b的全部字符转换为b
    for (int i=1;i<=m;i++)
    {   for (int j=1;j<=n;j++)
        {   if (a[i-1]==b[j-1])
                dp[i][j]=dp[i-1][j-1];
            else
                dp[i][j]=min(min(dp[i-1][j],dp[i][j-1]),dp[i-1][j-1])+1;
        }
    }
    return dp[m][n];
}
```

【算法分析】时间复杂度为$O(m\times n)$，空间复杂度为$O(m\times n)$。